MS

SSN

```
# Data Loading, Preparation & Exploration

# install.packages("forecast")
# install.packages("smooth")
# install.packages("fpp3")
# install.packages("prophet")
#install.packages("future")

library(fpp3)
```

```
-- Attaching packages --------------------------------------- fpp3 1.0.3 --

v tibble      3.3.1     v tsibble     1.2.0
v dplyr       1.2.0     v tsibbledata 0.4.1
v tidyr       1.3.2     v ggtime      0.2.0
v lubridate   1.9.5     v feasts      0.5.0
v ggplot2     4.0.2     v fable       0.5.0

-- Conflicts ------------------------------------------ fpp3_conflicts --
x lubridate::date()    masks base::date()
x dplyr::filter()      masks stats::filter()
x tsibble::intersect() masks base::intersect()
x tsibble::interval()  masks lubridate::interval()
x dplyr::lag()         masks stats::lag()
x tsibble::setdiff()   masks base::setdiff()
x tsibble::union()     masks base::union()
```

```
library(smooth) # fro msarima(), sarima() and adam()
```

```
Loading required package: greybox
```

```
Package "greybox", v2.0.8 loaded.


Attaching package: 'greybox'

The following objects are masked from 'package:fabletools':

    MAE, MAPE, MASE, ME, MPE, MSE, RMSSE

The following object is masked from 'package:tsibble':

    measures

The following object is masked from 'package:lubridate':

    hm

The following object is masked from 'package:tidyr':

    spread

This is package "smooth", v4.3.1
```

```
library(forecast) # for tbats(),  mstl()
library(tsibble)
library(dplyr)
library(prophet)  # prophet()
```

```
Loading required package: Rcpp

Loading required package: rlang
```

```
library(fable)
library(fabletools)
library(dplyr)
library(lubridate)
library(zoo)          # na.approx() for gap interpolation
```

```
Attaching package: 'zoo'

The following object is masked from 'package:tsibble':

    index

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric
```

```
library(prophet)
library(fable)
library(fabletools)
library(future)
library(future.apply)
library(ggplot2)
library(tidyr)
library(gridExtra)
```

```
Attaching package: 'gridExtra'

The following object is masked from 'package:dplyr':

    combine
```

```
suppressWarnings(library(fpp3))
suppressWarnings(library(smooth))
```

Traffic flow data

```
traffic_raw =  read.csv("traffic_weather_full2020.csv", sep = ",", header = T)

# column names
names(traffic_raw)
```

```
 [1] "X5.Minutes"  "Flow"        "X.mph."      "week_day"    "holidays"
 [6] "Hour"        "Day"         "Time"        "Temperature" "Dew.Point"
[11] "Humidity"    "Wind"        "Wind.Speed"  "Wind.Gust"   "Pressure"
[16] "Precip."     "Condition"
```

```
# rows
head(traffic_raw)
```

```
     X5.Minutes Flow X.mph. week_day holidays          Hour      Day  Time
1 2/1/2020 0:00  338   66.0        6        1 2/1/2020 0:00 2/1/2020 12:00
2 2/1/2020 0:05  360   65.5        6        1 2/1/2020 0:05 2/1/2020 12:00
3 2/1/2020 0:10  347   65.2        6        1 2/1/2020 0:10 2/1/2020 12:00
4 2/1/2020 0:15  337   65.4        6        1 2/1/2020 0:15 2/1/2020 12:00
5 2/1/2020 0:20  302   66.3        6        1 2/1/2020 0:20 2/1/2020 12:00
6 2/1/2020 0:25  282   65.3        6        1 2/1/2020 0:25 2/1/2020 12:00
  Temperature Dew.Point Humidity Wind Wind.Speed Wind.Gust Pressure Precip.
1        55 F      29 F       37  VAR          3         0    29.39       0
2        55 F      29 F       37  VAR          3         0    29.39       0
3        55 F      29 F       37  VAR          3         0    29.39       0
4        55 F      29 F       37  VAR          3         0    29.39       0
5        55 F      29 F       37  VAR          3         0    29.39       0
6        55 F      29 F       37  VAR          3         0    29.39       0
  Condition
1      Fair
2      Fair
3      Fair
4      Fair
5      Fair
6      Fair
```

```
str(traffic_raw)
```

```
'data.frame':   34823 obs. of  17 variables:
 $ X5.Minutes : chr  "2/1/2020 0:00" "2/1/2020 0:05" "2/1/2020 0:10" "2/1/2020 0:15" ...
 $ Flow       : int  338 360 347 337 302 282 302 277 273 290 ...
 $ X.mph.     : num  66 65.5 65.2 65.4 66.3 65.3 66.2 66.2 65.6 66.2 ...
 $ week_day   : int  6 6 6 6 6 6 6 6 6 6 ...
 $ holidays   : int  1 1 1 1 1 1 1 1 1 1 ...
 $ Hour       : chr  "2/1/2020 0:00" "2/1/2020 0:05" "2/1/2020 0:10" "2/1/2020 0:15" ...
 $ Day        : chr  "2/1/2020" "2/1/2020" "2/1/2020" "2/1/2020" ...
 $ Time       : chr  "12:00" "12:00" "12:00" "12:00" ...
```

```
 $ Temperature: chr  "55 F" "55 F" "55 F" "55 F" ...
 $ Dew.Point  : chr  "29 F" "29 F" "29 F" "29 F" ...
 $ Humidity   : num  37 37 37 37 37 37 37 37 37 37 ...
 $ Wind       : chr  "VAR" "VAR" "VAR" "VAR" ...
 $ Wind.Speed : chr  "3" "3" "3" "3" ...
 $ Wind.Gust  : chr  "0" "0" "0" "0" ...
 $ Pressure   : num  29.4 29.4 29.4 29.4 29.4 ...
 $ Precip.    : num  0 0 0 0 0 0 0 0 0 0 ...
 $ Condition  : chr  "Fair" "Fair" "Fair" "Fair" ...
```

```r
# See what's actually in the "5 Minutes" column
head(traffic_raw$X5.Minutes, 5)
```

```
[1] "2/1/2020 0:00" "2/1/2020 0:05" "2/1/2020 0:10" "2/1/2020 0:15"
[5] "2/1/2020 0:20"
```

```r
# What is the timestamp column called?
names(traffic_raw)
```

```
 [1] "X5.Minutes"  "Flow"        "X.mph."      "week_day"    "holidays"
 [6] "Hour"        "Day"         "Time"        "Temperature" "Dew.Point"
[11] "Humidity"    "Wind"        "Wind.Speed"  "Wind.Gust"   "Pressure"
[16] "Precip."     "Condition"
```

```r
# First 10 timestamps exactly as they appear in the file
head(traffic_raw$X5.Minutes, 10)
```

```
 [1] "2/1/2020 0:00" "2/1/2020 0:05" "2/1/2020 0:10" "2/1/2020 0:15"
 [5] "2/1/2020 0:20" "2/1/2020 0:25" "2/1/2020 0:30" "2/1/2020 0:35"
 [9] "2/1/2020 0:40" "2/1/2020 0:45"
```

```r
# Last 10 timestamps
tail(traffic_raw$X5.Minutes, 10)
```

```
 [1] "5/31/2020 23:10" "5/31/2020 23:15" "5/31/2020 23:20" "5/31/2020 23:25"
 [5] "5/31/2020 23:30" "5/31/2020 23:35" "5/31/2020 23:40" "5/31/2020 23:45"
 [9] "5/31/2020 23:50" "5/31/2020 23:55"
```

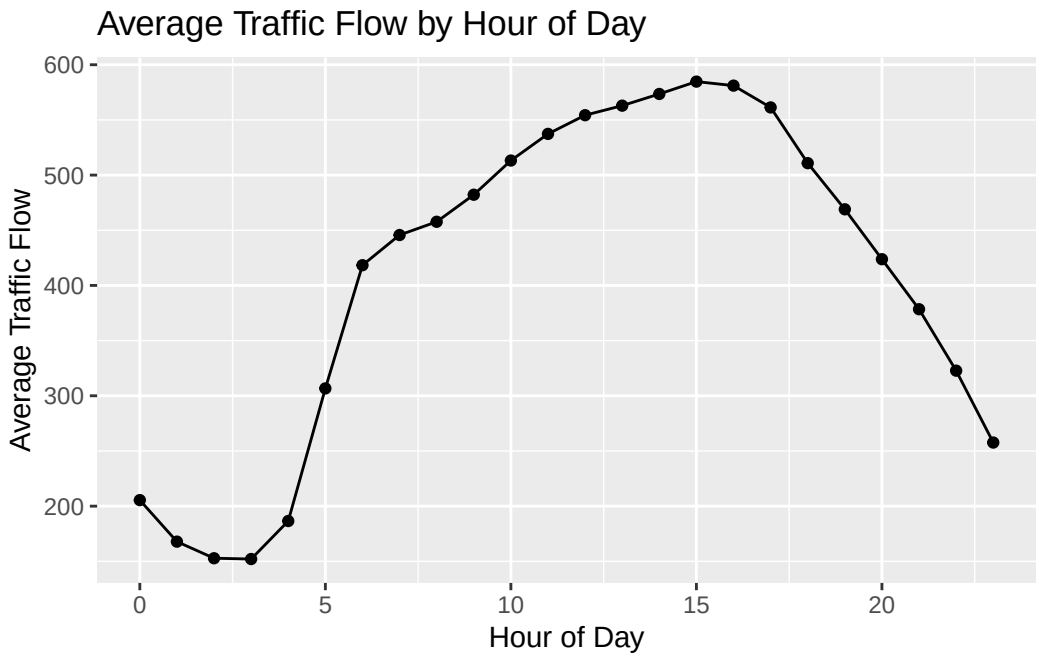
```r
# How many rows?
nrow(traffic_raw)
```

```
[1] 34823
```

```r
# Are there duplicated timestamps?
sum(duplicated(traffic_raw$X5.Minutes))
```

```
[1] 0
```

```r
traffic_raw %>%
  mutate(
    datetime = mdy_hm(X5.Minutes, tz = "UTC"),
    hour = hour(datetime)
  ) %>%
  group_by(hour) %>%
  summarise(avg_flow = mean(Flow), .groups = 'drop') %>%
  #arrange(desc(avg_flow)) %>%
  #head(10)
  ggplot(aes(x = hour, y = avg_flow)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Average Traffic Flow by Hour of Day",
    x = "Hour of Day",
    y = "Average Traffic Flow"
  )
```




```
ggsave("average_traffic_by_hour.pdf", width = 7, height = 5)
```

The plot illustrates the average traffic flow across of the hours of the day. The traffic volume gradually increases from early morning around 5 AM, peaking in the late afternoon to early evening (around 7 PM), and subsequently decreases during the night.

```
# Preparing the data
traffic_clean <- traffic_raw %>%
  mutate(datetime = mdy_hm(X5.Minutes, tz = "UTC")) %>%
  select(datetime, Flow, holidays) %>%
  rename(Traffic_flow = Flow)

head(traffic_clean)
```

```
             datetime Traffic_flow holidays
1 2020-02-01 00:00:00          338        1
2 2020-02-01 00:05:00          360        1
3 2020-02-01 00:10:00          347        1
4 2020-02-01 00:15:00          337        1
```

```
5 2020-02-01 00:20:00          302        1
6 2020-02-01 00:25:00          282        1
```

```
dim(traffic_clean)
```

```
[1] 34823     3
```

```
# Which timestamps are missing?
traffic_clean %>%
  mutate(dt_diff = datetime - lag(datetime)) %>%
  filter(dt_diff != as.difftime(5, units = "mins"))
```

```
             datetime Traffic_flow holidays dt_diff
1 2020-03-08 03:00:00          223        1 65 mins
2 2020-03-21 01:00:00          154        1 65 mins
3 2020-04-02 10:00:00          384        0 10 mins
```

Training data subset from April 1(2020) to May 31(2020)

```
# Training data subset: April 1 - May 31, 2020
traffic_subset <- traffic_clean %>%
  filter(
    as.Date(datetime) >= as.Date("2020-04-01"),
    as.Date(datetime) <= as.Date("2020-05-31")
  ) %>%
  as_tsibble(index = datetime)

traffic_subset %>% scan_gaps()
```

```
# A tsibble: 1 x 1 [5m] <UTC>
  datetime
  <dttm>
1 2020-04-02 09:55:00
```

Fill missing timestamps and linearly interpolate Traffic_flow (as an integer count)

```
traffic_subset %>%
  fill_gaps() %>%
  filter(
    datetime >= as.POSIXct("2020-04-01 09:40:00", tz = "UTC"),
    datetime <= as.POSIXct("2020-04-01 10:10:00", tz = "UTC")
  )
```

```
# A tsibble: 7 x 3 [5m] <UTC>
  datetime            Traffic_flow holidays
  <dttm>                     <int>    <int>
1 2020-04-01 09:40:00          436        0
2 2020-04-01 09:45:00          417        0
3 2020-04-01 09:50:00          411        0
4 2020-04-01 09:55:00          400        0
5 2020-04-01 10:00:00          408        0
6 2020-04-01 10:05:00          452        0
7 2020-04-01 10:10:00          387        0
```

```
# Linear Interpolation
#Fill missing timestamp and interpolate as an integer count

traffic_subset <- traffic_subset %>%
  fill_gaps() %>%
  mutate(
    Traffic_flow = as.integer(round(  ### We ant integer!
      na.approx(Traffic_flow, na.rm = FALSE)
    ))
  )

traffic_subset %>%
  filter(
    datetime >= as.POSIXct("2020-04-02 09:40:00", tz = "UTC"),
    datetime <= as.POSIXct("2020-04-02 10:10:00", tz = "UTC")
  )
```

```
# A tsibble: 7 x 3 [5m] <UTC>
  datetime            Traffic_flow holidays
```

```
  <dttm>                    <int>    <int>
1 2020-04-02 09:40:00         400        0
2 2020-04-02 09:45:00         417        0
3 2020-04-02 09:50:00         431        0
4 2020-04-02 09:55:00         408       NA
5 2020-04-02 10:00:00         384        0
6 2020-04-02 10:05:00         397        0
7 2020-04-02 10:10:00         377        0
```

```
traffic_subset %>% scan_gaps()
```

```
# A tsibble: 0 x 1 [?] <UTC>
# i 1 variable: datetime <dttm>
```

Aggregate the 5-min data to obtain 10-min dataset

```
traffic_10min <- traffic_subset %>%
  index_by(datetime_10 = ~ floor_date(.x, "10 minutes")) %>%
  summarise(Traffic_flow = sum(Traffic_flow, na.rm = TRUE)) %>%
  rename(datetime = datetime_10)

cat("Total 10-min observations:", nrow(traffic_10min), "\n")
```

```
Total 10-min observations: 8784
```

```
cat("Any NAs?", sum(is.na(traffic_10min$Traffic_flow)), "\n")
```

```
Any NAs? 0
```

Observe how the aggregation is done

```
traffic_subset %>%
  filter(datetime >= as.POSIXct("2020-04-2 09:45:00", tz = "UTC"))
```

```
# A tsibble: 17,163 x 3 [5m] <UTC>
   datetime            Traffic_flow holidays
   <dttm>                     <int>    <int>
 1 2020-04-02 09:45:00          417        0
 2 2020-04-02 09:50:00          431        0
 3 2020-04-02 09:55:00          408       NA
 4 2020-04-02 10:00:00          384        0
 5 2020-04-02 10:05:00          397        0
 6 2020-04-02 10:10:00          377        0
 7 2020-04-02 10:15:00          389        0
 8 2020-04-02 10:20:00          389        0
 9 2020-04-02 10:25:00          392        0
10 2020-04-02 10:30:00          423        0
# i 17,153 more rows
```

```
traffic_10min %>%
   filter(datetime >= as.POSIXct("2020-04-2 09:45:00", tz = "UTC"))
```

```
# A tsibble: 8,581 x 2 [10m] <UTC>
   datetime            Traffic_flow
   <dttm>                     <int>
 1 2020-04-02 09:50:00          839
 2 2020-04-02 10:00:00          781
 3 2020-04-02 10:10:00          766
 4 2020-04-02 10:20:00          781
 5 2020-04-02 10:30:00          873
 6 2020-04-02 10:40:00          858
 7 2020-04-02 10:50:00          834
 8 2020-04-02 11:00:00          786
 9 2020-04-02 11:10:00          845
10 2020-04-02 11:20:00          862
# i 8,571 more rows
```

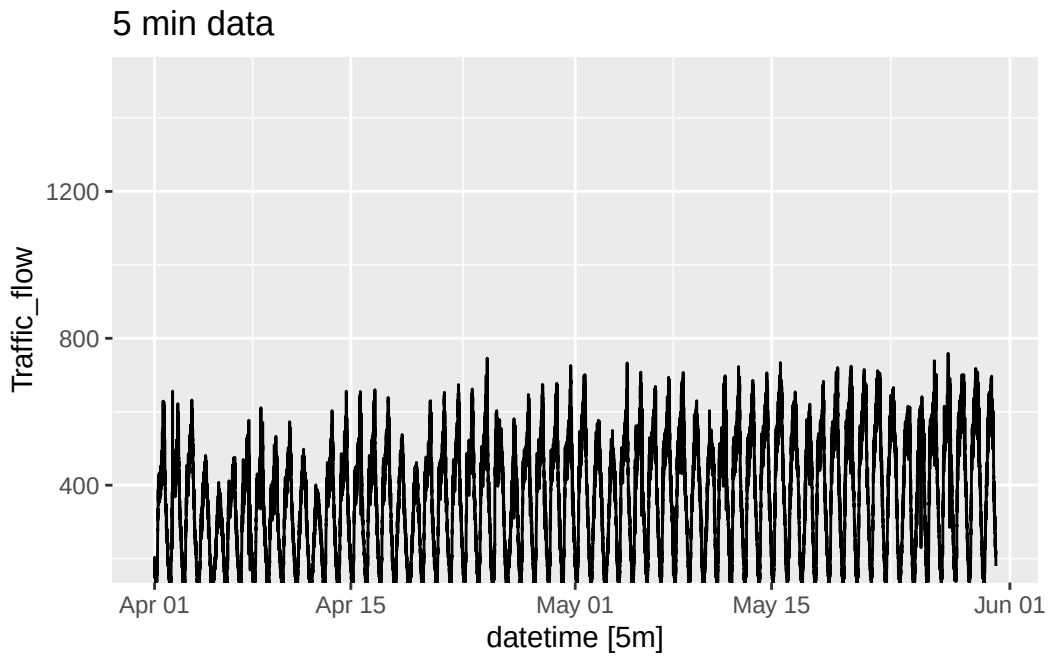
```
cat("Total 10-min observations:", nrow(traffic_10min), "\n") # 8784
```

```
Total 10-min observations: 8784
```

Plotting 5mins and 10 mins(aggregated) data for 2 weeks

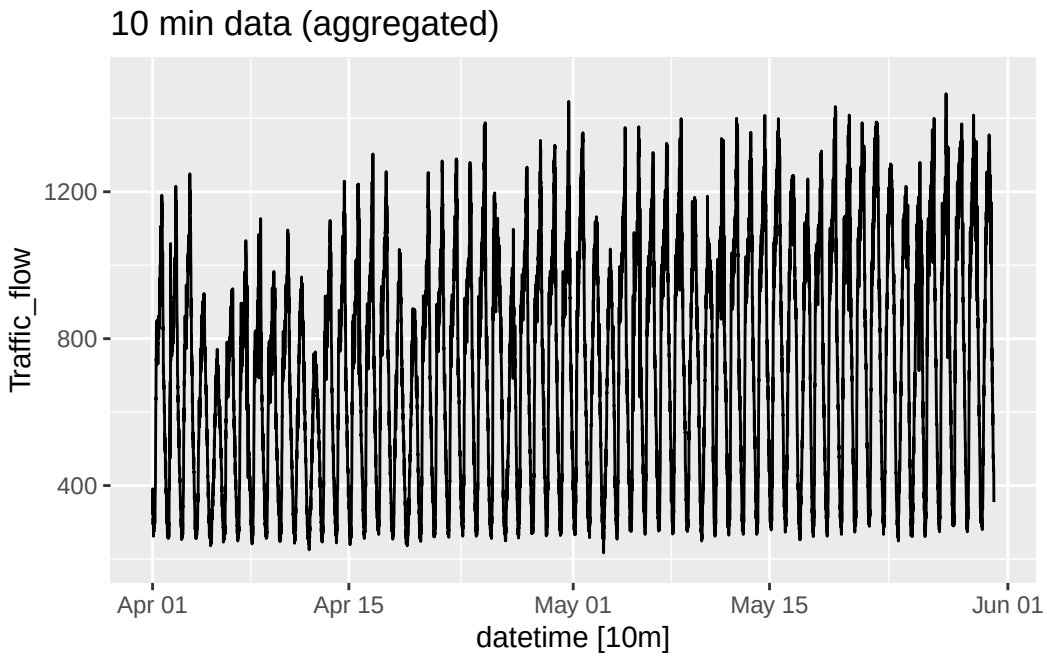
```r
p1 <- traffic_subset %>%
  filter(
    datetime >= as.POSIXct("2020-04-01 00:00:00", tz = "UTC"),
    datetime <= as.POSIXct("2020-05-31 00:00:00", tz = "UTC")
  ) %>%
  autoplot(Traffic_flow) + coord_cartesian(ylim = c(200, 1500)) +
  labs(title = "5 min data")

print(p1)
```



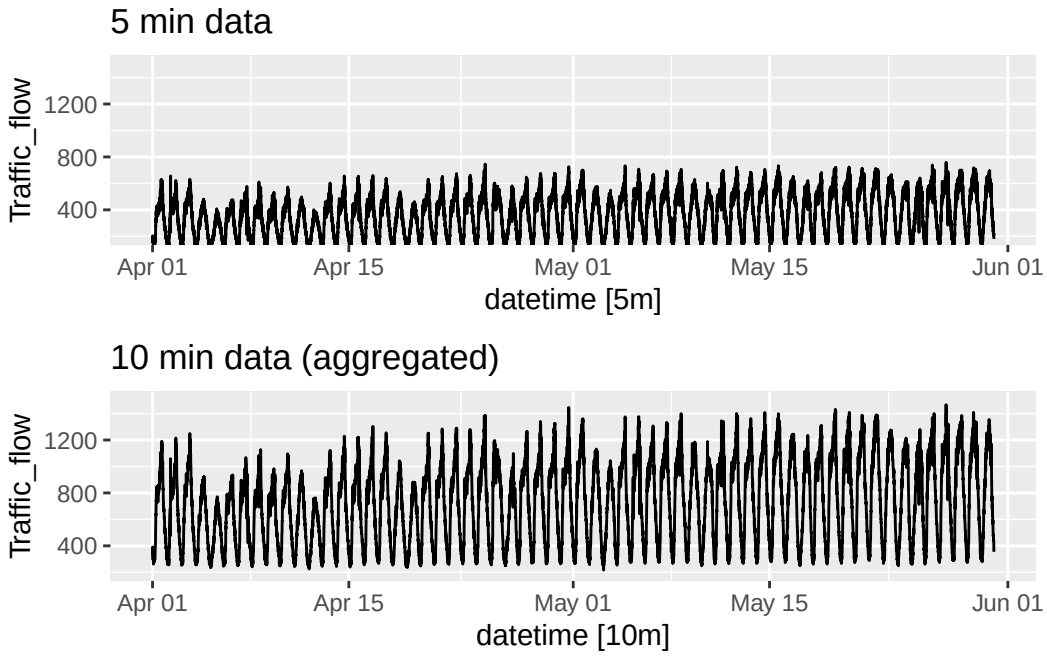

```r
p2 <- # Full range plot (Apr 1 - May 31), y-axis clipped for readability
traffic_10min %>%
  filter(
    datetime >= as.POSIXct("2020-04-01 00:00:00", tz = "UTC"),
    datetime <= as.POSIXct("2020-05-31 00:00:00", tz = "UTC")
  ) %>%
  autoplot(Traffic_flow) +
  coord_cartesian(ylim = c(200, 1500)) +
  labs(title = "10 min data (aggregated)")

print(p2)
```




```
ggsave("traffic_plot_p1.pdf", plot = p1, width = 7, height = 5)
ggsave("traffic_plot_p2.pdf", plot = p2, width = 7, height = 5)

p3 <- gridExtra::grid.arrange(p1, p2, nrow = 2)
```




```
#ggsave("traffic_plot_p3.pdf", plot = p3, width = 7, height = 5)
```

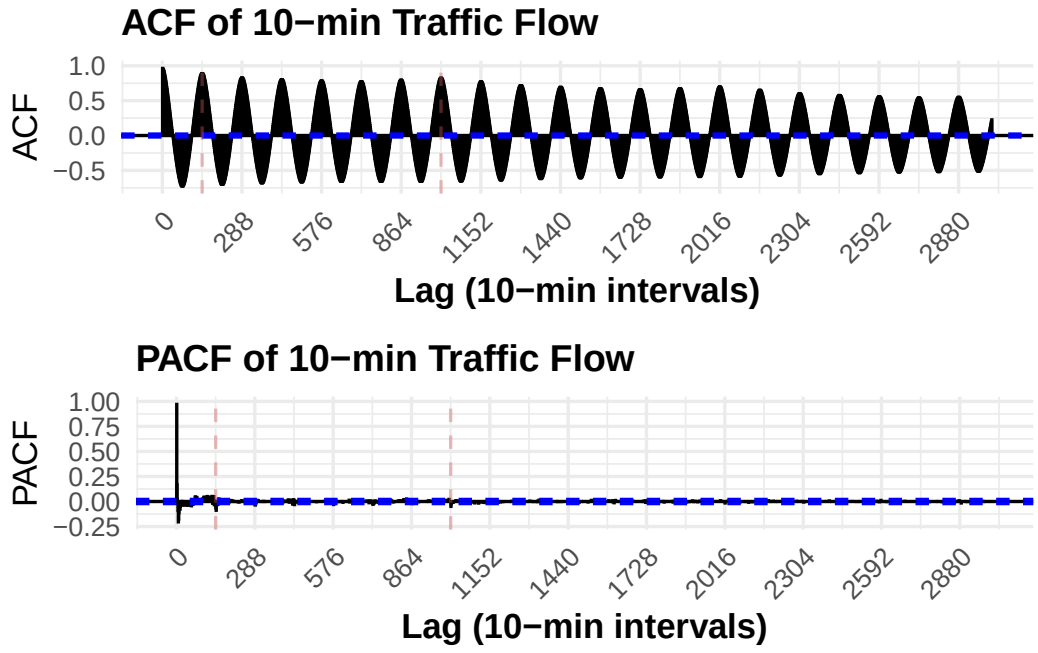
ACF and PACF plots

```
# ACF and PACF plots
p1 <- traffic_10min %>%
  ACF(Traffic_flow, lag_max = 3000) %>%
  autoplot() +
  labs(title = "ACF of 10-min Traffic Flow", x = "Lag (10-min intervals)", y = "ACF") +
  scale_x_continuous(breaks = seq(0, 3000, by = 288)) +   # every ~2 days (144 = 1 day)
  geom_vline(xintercept = c(144, 1008), linetype = "dashed", color = "indianred", alpha = 0.4
  theme_minimal(base_size = 13) +
  theme(
    axis.text.x = element_text(size = 10, angle = 45, hjust = 1),
    axis.title.x = element_text(size = 13, face = "bold"),
    plot.title  = element_text(size = 14, face = "bold")
  )

p2 <- traffic_10min %>%
  PACF(Traffic_flow, lag_max = 3000) %>%
```

```
  autoplot() +
  labs(title = "PACF of 10-min Traffic Flow", x = "Lag (10-min intervals)", y = "PACF") +
  scale_x_continuous(breaks = seq(0, 3000, by = 288)) +
  geom_vline(xintercept = c(144, 1008), linetype = "dashed", color = "indianred", alpha = 0.4
  theme_minimal(base_size = 13) +
  theme(
    axis.text.x = element_text(size = 10, angle = 45, hjust = 1),
    axis.title.x = element_text(size = 13, face = "bold"),
    plot.title  = element_text(size = 14, face = "bold")
  )

gridExtra::grid.arrange(p1, p2, nrow = 2, heights = c(1, 1))
```




```
pdf("ACF_PACF_10min_original.pdf", width = 14, height = 9)  # TODO: re-enable for report expo
grid.arrange(p1, p2, nrow = 2)
dev.off()
```

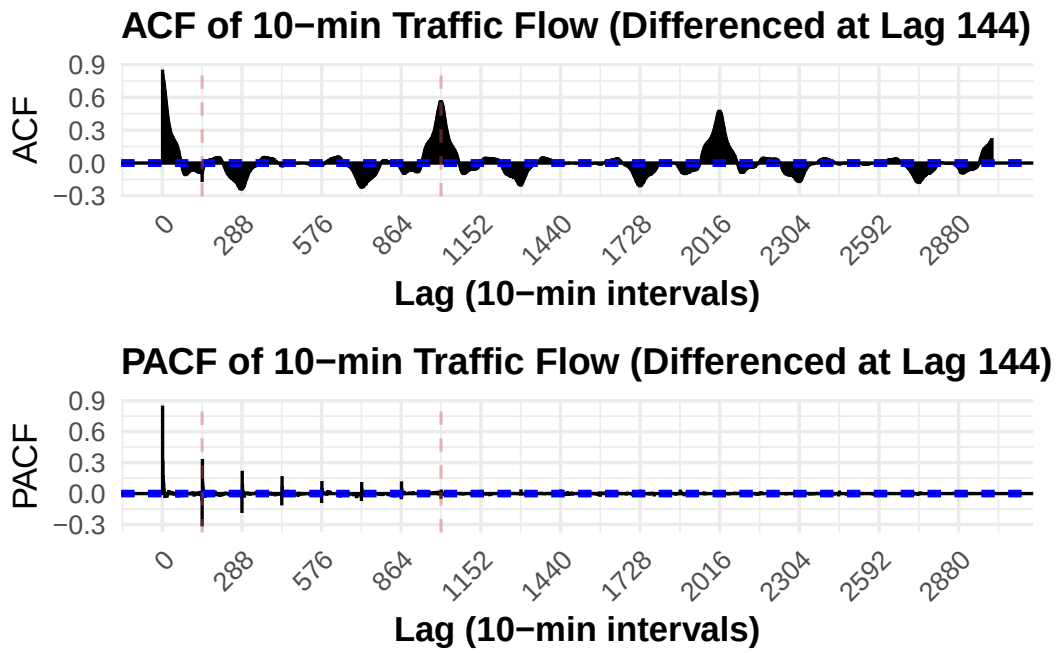
```
pdf
  2
```

ACF and PACF after differencing (removing daily seasonality , shows weekly seasonality)

```
p1<- traffic_10min %>%
  mutate(diff_144 = difference(Traffic_flow, lag = 144)) %>%
  ACF(diff_144, lag_max = 3000) %>%
  autoplot() +
  labs(title = "ACF of 10-min Traffic Flow (Differenced at Lag 144)", x = "Lag (10-min interv
  scale_x_continuous(breaks = seq(0, 3000, by = 288)) +   # every ~2 days (144 = 1 day)
  geom_vline(xintercept = c(144, 1008), linetype = "dashed", color = "indianred", alpha = 0.4
  theme_minimal(base_size = 13) +
  theme(
    axis.text.x = element_text(size = 10, angle = 45, hjust = 1),
    axis.title.x = element_text(size = 13, face = "bold"),
    plot.title  = element_text(size = 14, face = "bold")
  )

p2<- traffic_10min %>%
  mutate(diff_144 = difference(Traffic_flow, lag = 144)) %>%
  PACF(diff_144, lag_max = 3000) %>%
  autoplot() +
  labs(title = "PACF of 10-min Traffic Flow (Differenced at Lag 144)", x = "Lag (10-min inter
  scale_x_continuous(breaks = seq(0, 3000, by = 288)) +
  geom_vline(xintercept = c(144, 1008), linetype = "dashed", color = "indianred", alpha = 0.4
  theme_minimal(base_size = 13) +
  theme(
    axis.text.x = element_text(size = 10, angle = 45, hjust = 1),
    axis.title.x = element_text(size = 13, face = "bold"),
    plot.title  = element_text(size = 14, face = "bold")
  )

gridExtra::grid.arrange(p1, p2, nrow = 2)
```




```
pdf("ACF_PACF_10min_after_diff.pdf", width = 10, height = 8)  # TODO: re-enable for report ex
grid.arrange(p1, p2, nrow = 2)
dev.off()
```

```
pdf
  2
```

Train Test Split

```
# Train/test split at May 10
traffic_train <- traffic_10min %>%
  filter(as.Date(datetime) <= as.Date("2020-05-10"))

traffic_test <- traffic_10min %>%
  filter(as.Date(datetime) > as.Date("2020-05-10"))

tail(traffic_train)
```

```
# A tsibble: 6 x 2 [10m] <UTC>
```

```
  datetime            Traffic_flow
  <dttm>                     <int>
1 2020-05-10 23:00:00          562
2 2020-05-10 23:10:00          542
3 2020-05-10 23:20:00          523
4 2020-05-10 23:30:00          464
5 2020-05-10 23:40:00          507
6 2020-05-10 23:50:00          471
```

```
head(traffic_test)
```

```
# A tsibble: 6 x 2 [10m] <UTC>
  datetime            Traffic_flow
  <dttm>                     <int>
1 2020-05-11 00:00:00          423
2 2020-05-11 00:10:00          412
3 2020-05-11 00:20:00          386
4 2020-05-11 00:30:00          381
5 2020-05-11 00:40:00          361
6 2020-05-11 00:50:00          327
```

```
cat("Test days implied:", nrow(traffic_test) / 144, "\n")
```

```
Test days implied: 21
```

```
cat("Total 10-min observations:", nrow(traffic_10min), "\n")
```

```
Total 10-min observations: 8784
```

```
cat("Any NAs?", sum(is.na(traffic_10min$Traffic_flow)), "\n")
```

```
Any NAs? 0
```

```
cat("Train observations:", nrow(traffic_train), "\n")
```

```
Train observations: 5760
```

```r
cat("Test observations:", nrow(traffic_test), "\n")
```

```
Test observations: 3024
```

```r
y_train <- traffic_train$Traffic_flow
```

Actual matrix

```r
h       <- 144
n_train <- nrow(traffic_train)
n_test  <- nrow(traffic_test)
y_full  <- traffic_10min$Traffic_flow

actual_matrix_full <- matrix(NA, nrow = n_test, ncol = h)
for (i in 1:n_test) {
  test_start <- n_train + i
  test_end   <- test_start + h - 1
  if (test_end <= length(y_full)) {
    actual_matrix_full[i, ] <- y_full[test_start:test_end]
  }
}

cat("h =", h, " n_train =", n_train, " n_test =", n_test, "\n")
```

```
h = 144  n_train = 5760  n_test = 3024
```

```r
cat("actual_matrix_full dimensions:", dim(actual_matrix_full), "\n")
```

```
actual_matrix_full dimensions: 3024 144
```

MSARIMA - using identification method

Identifying the daily component

```
# # 24h * 60min = 1440 min/day -> 1440/10min = 144 obs/day
# # lags = c(1, 144): no weekly component yet
# # non-seasonal (0,1,1): d=1 for trend, MA(1) as starting point
# # daily seasonal (P,D,Q) varies across candidates
#
# y_train <- traffic_train$Traffic_flow
#
# # Candidate 1: simplest - just seasonal AR(1) at lag 144
# fit_d1 <-
#   msarima(
#     y_train,
#     lags   = c(1, 144),
#     orders = list(
#       ar = c(0, 1),
#       i  = c(1, 0),
#       ma = c(1, 0),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
#
# # Candidate 2: seasonal MA(1) at lag 144
# fit_d2 <- msarima(
#     y_train,
#     lags   = c(1, 144),
#     orders = list(
#       ar = c(0, 0),
#       i  = c(1, 0),
#       ma = c(1, 1),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
# # Candidate 3: seasonal AR(1) + MA(1) at lag 144
# fit_d3 <-msarima(
```

```
#     y_train,
#     lags   = c(1, 144),
#     orders = list(
#       ar = c(0, 1),
#       i  = c(1, 0),
#       ma = c(1, 1),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
# # Candidate 4: seasonal AR(1) + MA(2) - the paper's final spec uses
# fit_d4 <- msarima(
#     y_train,
#     lags   = c(1, 144),
#     orders = list(
#       ar = c(0, 1),
#       i  = c(1, 0),
#       ma = c(1, 2),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
# cat("Candidate 1 - (0,1,1)(1,0,0)[144] MAE:", accuracy(fit_d1)["MAE"], "\n")
# cat("Candidate 2 - (0,1,1)(0,0,1)[144] MAE:", accuracy(fit_d2)["MAE"], "\n")
# cat("Candidate 3 - (0,1,1)(1,0,1)[144] MAE:", accuracy(fit_d3)["MAE"], "\n")
# cat("Candidate 4 - (0,1,1)(1,0,2)[144] MAE:", accuracy(fit_d4)["MAE"], "\n")
```

```
# # Plotting residual ACF at multiples of 144 for each candidate
#
# par(mfrow = c(2, 2))
#
# acf(residuals(fit_d1), lag.max = 1500,
#     main = "Residual ACF: (0,1,1)(1,0,0)[144]")
# abline(v = seq(144, 1440, by = 144), col = "indianred", lty = 2)
#
# acf(residuals(fit_d2), lag.max = 1500,
```

```
#     main = "Residual ACF: (0,1,1)(0,0,1)[144]")
# abline(v = seq(144, 1440, by = 144), col = "indianred", lty = 2)
#
# acf(residuals(fit_d3), lag.max = 1500,
#     main = "Residual ACF: (0,1,1)(1,0,1)[144]")
# abline(v = seq(144, 1440, by = 144), col = "indianred", lty = 2)
#
# acf(residuals(fit_d4), lag.max = 1500,
#     main = "Residual ACF: (0,1,1)(1,0,2)[144]")
# abline(v = seq(144, 1440, by = 144), col = "indianred", lty = 2)
#
# par(mfrow = c(1, 1))
```

Identifying Weekly Component

```
# # Using confirmed daily spec (0,1,1)(1,0,1)[144]
# # Now add weekly candidates at lag 1008
# # We only have 5760 obs = 5760/1008  5.7 weeks.  (144 * 7days = 1008 obs)
# # So we can only observe weekly lags at 1008,2016,...
# # we don't have enough weeks for high orders so keep the orders low of weekly.
#
# # Candidate W1: weekly AR(1) only
# fit_w1 <-msarima(
#     y_train,
#     lags   = c(1, 144, 1008),
#     orders = list(
#       ar = c(0, 1, 1),
#       i  = c(1, 0, 0),
#       ma = c(1, 1, 0),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
#
# # Candidate W2: weekly MA(1) only
# fit_w2 <- msarima(
#     y_train,
```

```
#     lags   = c(1, 144, 1008),
#     orders = list(
#       ar = c(0, 1, 0),
#       i  = c(1, 0, 0),
#       ma = c(1, 1, 1),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
# # Candidate W3: weekly AR(1) + MA(1)
# fit_w3 <- msarima(
#     y_train,
#     lags   = c(1, 144, 1008),
#     orders = list(
#       ar = c(0, 1, 1),
#       i  = c(1, 0, 0),
#       ma = c(1, 1, 1),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
# # Candidate W4: weekly AR(1) + MA(2)
# fit_w4 <- msarima(
#     y_train,
#     lags   = c(1, 144, 1008),
#     orders = list(
#       ar = c(0, 1, 1),
#       i  = c(1, 0, 0),
#       ma = c(1, 1, 2),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
```

```
# # Candidate W5: weekly AR(2) + MA(2)
# fit_w5 <- msarima(
#     y_train,
#     lags   = c(1, 144, 1008),
#     orders = list(
#       ar = c(0, 1, 1),
#       i  = c(1, 0, 0),
#       ma = c(1, 2, 2),
#       initial = "backcasting",
#       bounds = "admissible",
#       loss = "likelihood",
#       ic = "BIC"
#     )
#   )
#
# cat("W1 - (0,1,1)(1,0,1)[144](1,0,0)[1008] MAE:", accuracy(fit_w1)["MAE"], "\n")
# cat("W2 - (0,1,1)(1,0,1)[144](0,0,1)[1008] MAE:", accuracy(fit_w2)["MAE"], "\n")
# cat("W3 - (0,1,1)(1,0,1)[144](1,0,1)[1008] MAE:", accuracy(fit_w3)["MAE"], "\n")
# cat("W4 - (0,1,1)(1,0,1)[144](1,0,2)[1008] MAE:", accuracy(fit_w4)["MAE"], "\n")
# cat("W5 - (0,1,1)(1,0,2)[144](1,0,2)[1008] MAE:", accuracy(fit_w5)["MAE"], "\n")
```

Residual ACF plots

```
# # Red  = daily lags  (144, 288, 432...)
# # Blue = weekly lags (1008, 2016...)
# par(mfrow = c(2, 2))
#
# acf(residuals(fit_w1), lag.max = 2500,
#     main = "W1: (1,0,1)[144](1,0,0)[1008]")
# abline(v = seq(144,  2500, by = 144),  col = "indianred",  lty = 2)
# abline(v = seq(1008, 2500, by = 1008), col = "darkblue", lty = 2)
#
# acf(residuals(fit_w2), lag.max = 2500,
#     main = "W2: (1,0,1)[144](0,0,1)[1008]")
# abline(v = seq(144,  2500, by = 144),  col = "indianred",  lty = 2)
# abline(v = seq(1008, 2500, by = 1008), col = "darkblue", lty = 2)
#
# acf(residuals(fit_w3), lag.max = 2500,
#     main = "W3: (1,0,1)[144](1,0,1)[1008]")
```

```
# abline(v = seq(144,  2500, by = 144),  col = "indianred",  lty = 2)
# abline(v = seq(1008, 2500, by = 1008), col = "darkblue", lty = 2)
#
# acf(residuals(fit_w4), lag.max = 2500,
#     main = "W4: (1,0,2)[144](1,0,2)[1008]")
# abline(v = seq(144,  2500, by = 144),  col = "indianred",  lty = 2)
# abline(v = seq(1008, 2500, by = 1008), col = "darkblue", lty = 2)
#
# par(mfrow = c(1, 1))
```

Msarima - using auto.msarima() from smooth package

```
time_auto <- system.time({
  fit_auto <- auto.msarima(
      y_train,
      lags   = c(1, 144, 1008),
      orders = list(
        ar = c(1, 1, 1),
        i  = c(1, 0, 0),
        ma = c(1, 2, 2)

      ),
      silent = TRUE,
      initial = c("backcasting"),
      bounds = "admissible",
      loss = "likelihood",
      ic = "BIC"
    )
})

# fit is same as in the paper with the paper
cat("auto.msarima() selected:", fit_auto$model, "\n")
```

```
auto.msarima() selected: SARIMA(0,1,1)[1](1,0,2)[144](1,0,2)[1008]
```

```
cat("Paper's spec:            mSARIMA(0,1,1)(1,0,2)[144](1,0,2)[1008]\n")
```

```
Paper's spec:            mSARIMA(0,1,1)(1,0,2)[144](1,0,2)[1008]
```

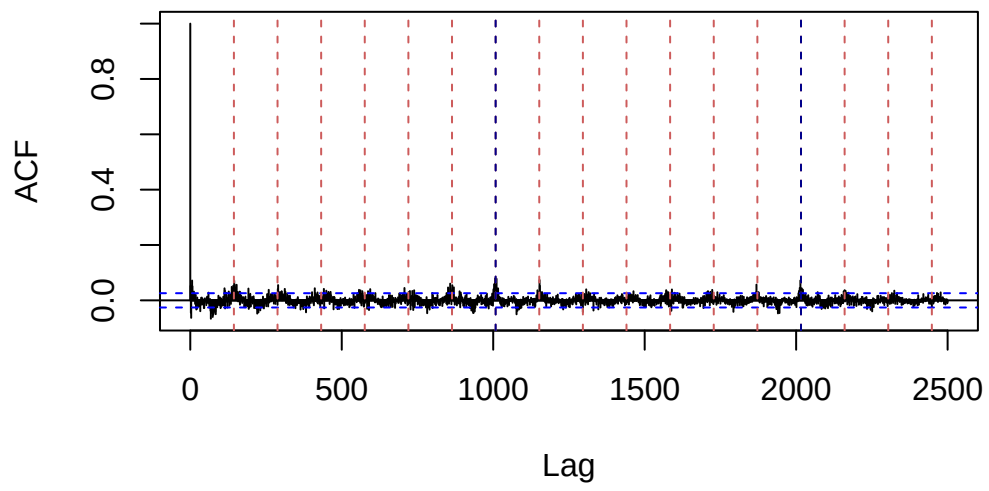
```r
cat("In-sample MAE - auto:  ", accuracy(fit_auto)["MAE"], "\n")
```

```
In-sample MAE - auto:   28.7057
```

```r
coef(fit_auto)
```

```
   theta1[1]    phi1[144]  theta1[144]  theta2[144]   phi1[1008] theta1[1008]
 -0.50394384   0.98210059  -0.87102111   0.01341205   0.25750251  -0.16151222
theta2[1008]
  0.06370042
```

```r
acf(residuals(fit_auto), lag.max = 2500, main = paste("auto.msarima residual ACF:", fit_auto$
abline(v = seq(144, 2500, 144), col = "indianred", lty = 2)
abline(v = seq(1008, 2500, 1008), col = "darkblue", lty = 2)
```



EXPAND WINDOW METHOD

```
#if (file.exists("fc_msarima_full.RData")) file.remove("fc_msarima_full.RData")
#if (file.exists("fc_sarima_full.RData")) file.remove("fc_sarima_full.RData")
#if (file.exists("fc_tbats_full.RData")) file.remove("fc_tbats_full.RData")
#if (file.exists("fc_mstl_full.RData")) file.remove("fc_mstl_full.RData")
#if (file.exists("fc_prophet_full.RData")) file.remove("fc_prophet_full.RData")
#if (file.exists("fc_dhr_full.RData")) file.remove("fc_dhr_full.RData")
```

```
FORECAST_ORIGINS <- 1000
```

```
#   Coverage & interval width
# actual, lower, upper are all (n_test x h) matrices, same shape as
# Coverage = proportion of actual values falling inside [lower bound, upper bound]
# Width    = average interval size (upper bound - lower bound)

coverage_width <- function(actual, lower, upper) {
  covered <- (actual >= lower) & (actual <= upper)
  list(
    coverage_overall = mean(covered, na.rm = TRUE),
    width_overall    = mean(upper - lower, na.rm = TRUE),
    coverage_by_h    = colMeans(covered, na.rm = TRUE),   # for the h vs coverage plot
    width_by_h       = colMeans(upper - lower, na.rm = TRUE)
  )
}
```

```
msarima_full_path <- "fc_msarima_full.RData"

if (file.exists(msarima_full_path)) {

  message("[cache hit] loading ", msarima_full_path, " - skipping mSARIMA expanding window lo
  load(msarima_full_path)

} else {

  n_workers <- max(1, parallel::detectCores() - 1)
  cat("Running mSARIMA expanding window (", FORECAST_ORIGINS, "refits) across", n_workers, "w

  plan(multisession, workers = n_workers)

  time_msarima_full <- system.time({
    results_msarima <- future_lapply(1:FORECAST_ORIGINS, function(i) {
```

```
    train_end_i <- n_train + i - 1
    y_train_i   <- y_full[1:train_end_i]

    tryCatch({
      fit_i <- smooth::msarima(
        y_train_i,
        lags   = c(1, 144, 1008),
        orders = list(
          ar = c(1, 1, 1),
          i  = c(1, 0, 0),
          ma = c(1, 2, 2)
        ),
        silent = TRUE
      )

      fc_i <- forecast::forecast(fit_i, h = h, interval = "parametric", level = 0.95)

      list(
        mean  = as.numeric(fc_i$mean),
        lower = as.numeric(as.matrix(fc_i$lower)[, 1]),
        upper = as.numeric(as.matrix(fc_i$upper)[, 1]),
        ok = TRUE, error = NA_character_
      )
    }, error = function(e) {
      list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
           ok = FALSE, error = conditionMessage(e))
    })

  }, future.seed = TRUE)
})

plan(sequential) # turns the parallelisation back off

fc_msarima_full       <- do.call(rbind, lapply(results_msarima, `[[`, "mean"))
fc_msarima_full_lower <- do.call(rbind, lapply(results_msarima, `[[`, "lower"))
fc_msarima_full_upper <- do.call(rbind, lapply(results_msarima, `[[`, "upper"))

n_failed <- sum(!sapply(results_msarima, `[[`, "ok"))
if (n_failed > 0) {
  cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
  failed_idx <- which(!sapply(results_msarima, `[[`, "ok"))
  cat("Failed origins:", paste(head(failed_idx, 10), collapse = ", "),
```

```
        if (length(failed_idx) > 10) "..." else "", "\n")
  } else {
    cat("All", FORECAST_ORIGINS, "origins fit successfully.\n")
  }

  cat("mSARIMA loop time:", round(time_msarima_full["elapsed"] / 60, 2), "minutes\n")

  save(fc_msarima_full, fc_msarima_full_lower, fc_msarima_full_upper,
       file = msarima_full_path)
  cat("mSARIMA forecasts + 95% PI saved.\n")
}
```

```
[cache hit] loading fc_msarima_full.RData - skipping mSARIMA expanding window loop
```

```
# mSARIMA MAE / RMSE
mae_msarima_full <- sapply(1:h, function(hh) {
  mean(abs(fc_msarima_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh]), na.rm = TRUE)
})

rmse_msarima_full <- sapply(1:h, function(hh) {
  sqrt(mean((fc_msarima_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh])^2, na.rm = T
})

cat("mSARIMA MAE: ", round(mean(mae_msarima_full), 3), "\n")
```

```
mSARIMA MAE:  99.491
```

```
cat("mSARIMA RMSE:", round(mean(rmse_msarima_full), 3), "\n")
```

```
mSARIMA RMSE: 121.168
```

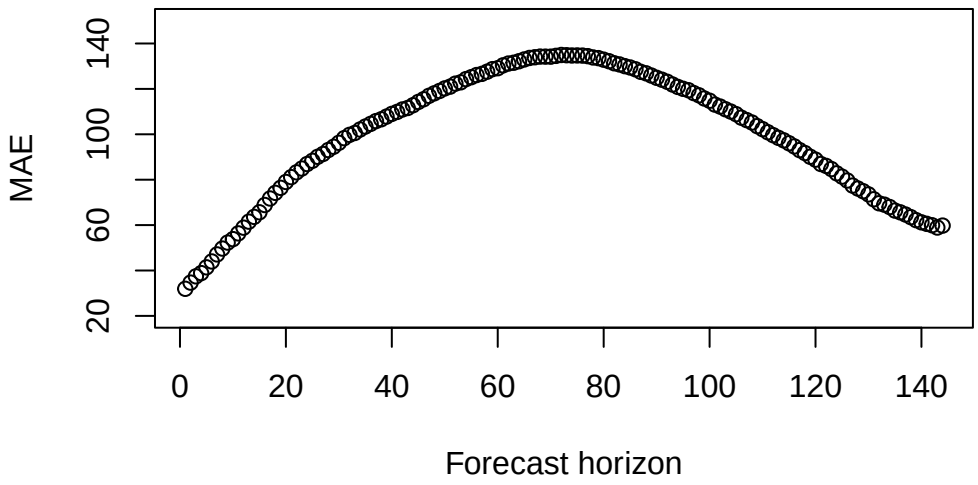
```
# mSARIMA 95% PI coverage & width
msarima_pi <- coverage_width(actual_matrix_full[1:FORECAST_ORIGINS, ], fc_msarima_full_lower
cat("mSARIMA 95% coverage:", round(msarima_pi$coverage_overall, 3), " (target 0.95)\n")
```

```
mSARIMA 95% coverage: 0.972  (target 0.95)
```

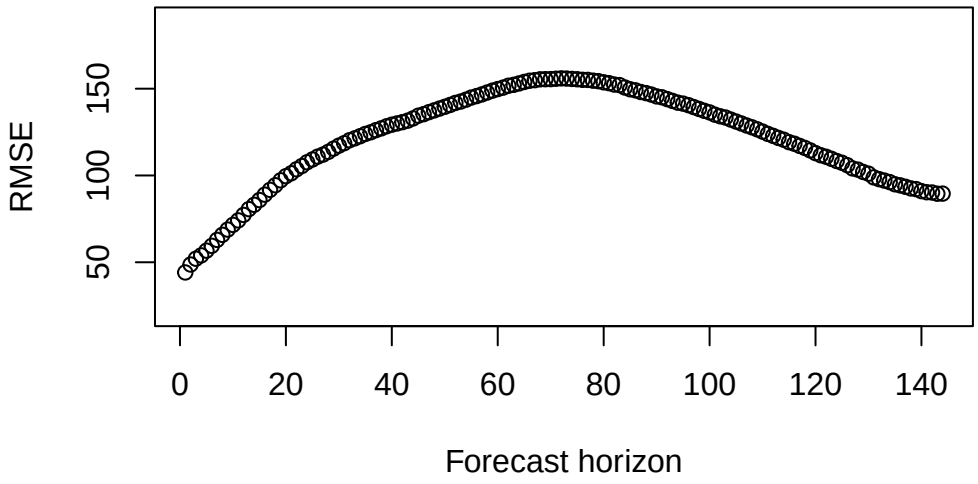
```r
cat("mSARIMA 95% width:   ", round(msarima_pi$width_overall, 2), "\n")
```

```
mSARIMA 95% width:    599.12
```

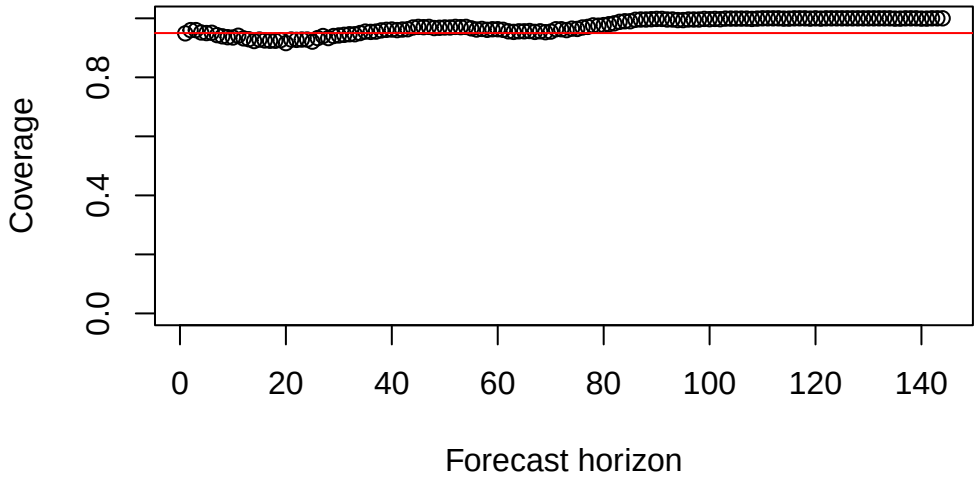
```r
# MAE / RMSE by horizon
plot(mae_msarima_full, ylim = c(20, 150), xlab = "Forecast horizon", ylab = "MAE")
```



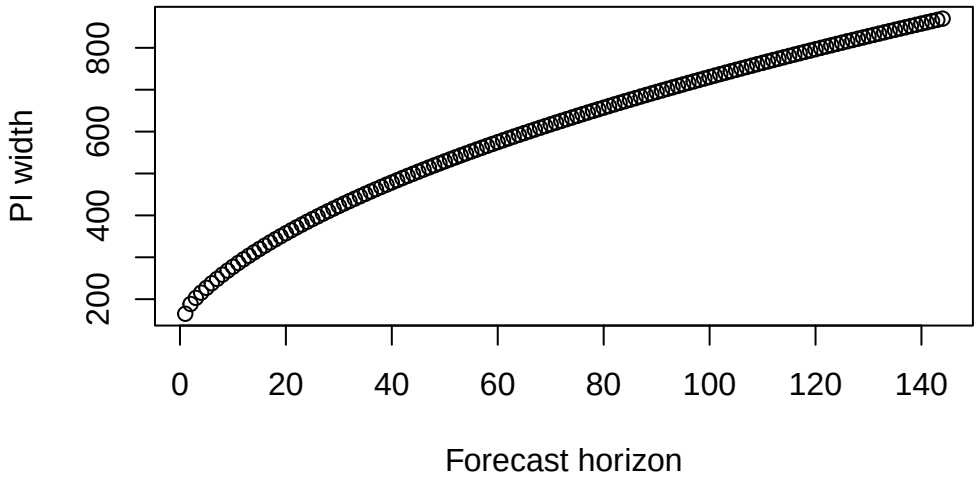

```r
plot(rmse_msarima_full, ylim = c(20, 190), xlab = "Forecast horizon", ylab = "RMSE")
```



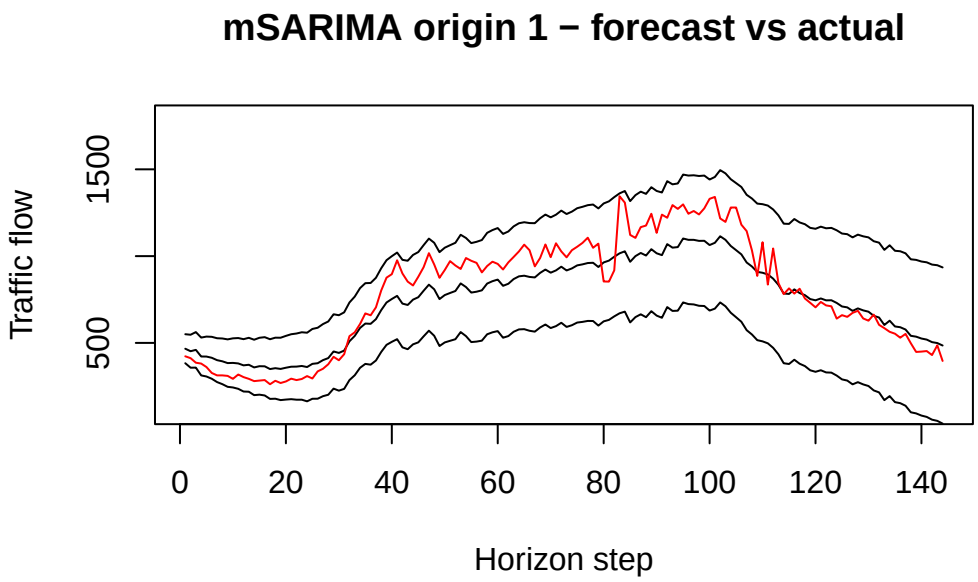

```r
# Coverage by horizon
plot(msarima_pi$coverage_by_h, ylim = c(0, 1), xlab = "Forecast horizon", ylab = "Coverage")
abline(h = 0.95, col = "red")
```



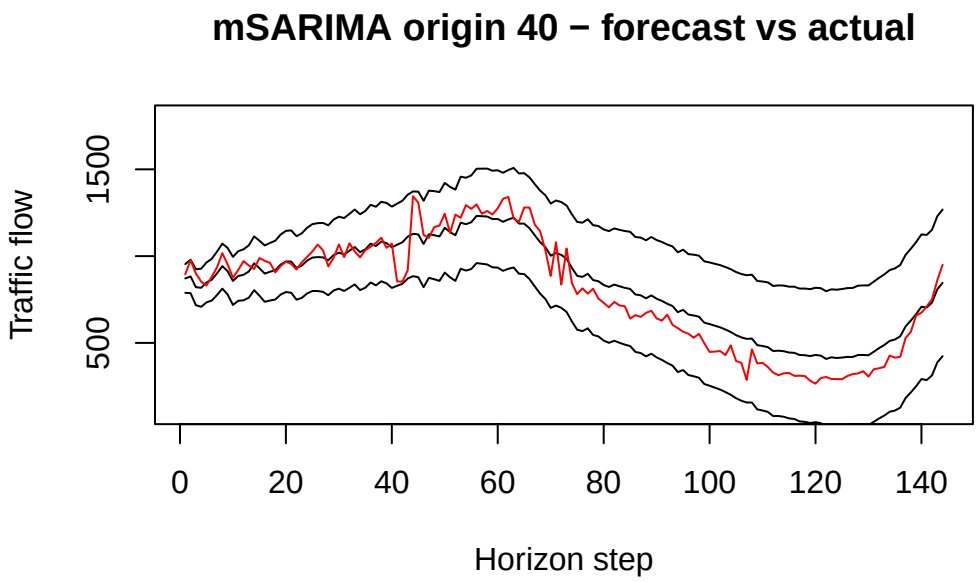
```r
# Width by horizon
plot(msarima_pi$width_by_h, xlab = "Forecast horizon", ylab = "PI width")
```




```r
# Sample forecast vs actual, for two origins
plot(fc_msarima_full[1, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "T
     main = "mSARIMA origin 1 - forecast vs actual")
lines(fc_msarima_full_lower[1, ])
lines(fc_msarima_full_upper[1, ])
lines(actual_matrix_full[1, ], col = "red")
```




```
plot(fc_msarima_full[40, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "
     main = "mSARIMA origin 40 - forecast vs actual")
lines(fc_msarima_full_lower[40, ])
lines(fc_msarima_full_upper[40, ])
lines(actual_matrix_full[40, ], col = "red")
```



SARIMA - single seasonality (daily)

```
# Drop the weekly seasonality from mSARIMA specifications

time_sarima <- system.time({
  fit_sarima <- msarima(
      y_train,
      lags   = c(1, 144),
      orders = list(
        ar = c(0, 1),
        i  = c(1, 0),
        ma = c(1, 2)
      )
  )
})

# Results
cat("MAE:         ", accuracy(fit_sarima)["MAE"], "\n")
```

```
MAE:          28.99531
```

```r
rmse_msarima <- sqrt(mean((y_train - fitted(fit_sarima))^2,
                          na.rm = TRUE))
cat("SARIMA:", rmse_msarima, "\n")
```

```
SARIMA: 41.92134
```

```r
cat("Fitting time:", time_sarima["elapsed"], "seconds\n")
```

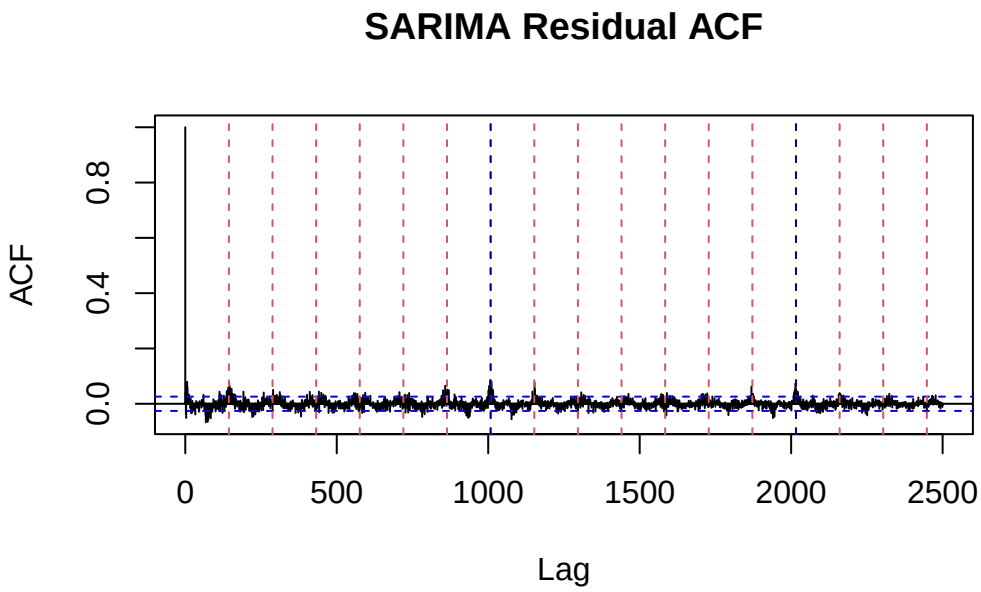
```
Fitting time: 0.321 seconds
```

```r
summary(fit_sarima)
```

```
Model estimated using msarima() function: SARIMA(0,1,1)[1](1,0,2)[144]
Response variable: data
Distribution used in the estimation: Normal
Loss function type: likelihood; Loss function value: 29691.26
Coefficients:
            Estimate Std. Error Lower 2.5% Upper 97.5%
theta1[1]    -0.4796     0.0113    -0.5016     -0.4575 *
phi1[144]     0.9905     0.0033     0.9840      0.9969 *
theta1[144]  -0.8744     0.0141    -0.9020     -0.8468 *
theta2[144]   0.0118     0.0135     0.3405      0.0384 *

Error standard deviation: 41.9395
Sample size: 5760
Number of estimated parameters: 5
Number of degrees of freedom: 5755
Information criteria:
     AIC     AICc      BIC     BICc
59392.53 59392.54 59425.82 59425.87
```

```r
# Residual ACF - only daily lags matter here (no weekly term)
acf(residuals(fit_sarima), lag.max = 2500,
    main = "SARIMA Residual ACF")
abline(v = seq(144,  2500, by = 144),  col = "indianred",  lty = 2)
abline(v = seq(1008, 2500, by = 1008), col = "darkblue", lty = 2)
```



Expand window method

```
sarima_full_path <- "fc_sarima_full.RData"

if (file.exists(sarima_full_path)) {

  message("[cache hit] loading ", sarima_full_path, " - skipping SARIMA expanding window loo
  load(sarima_full_path)

} else {

  n_workers <- max(1, parallel::detectCores() - 1)
  cat("Running SARIMA expanding window (", FORECAST_ORIGINS, "refits) across", n_workers, "w

  plan(multisession, workers = n_workers)

  time_sarima_full <- system.time({
    results_sarima <- future_lapply(1:FORECAST_ORIGINS, function(i) {

      train_end_i <- n_train + i - 1
```

```
    y_train_i   <- y_full[1:train_end_i]

    tryCatch({
      fit_i <- smooth::msarima(
        y_train_i,
        lags   = c(1, 144),
        orders = list(ar = c(0, 1), i = c(1, 0), ma = c(1, 2)),
        silent = TRUE
      )

      fc_i <- forecast::forecast(fit_i, h = h, interval = "parametric", level = 0.95)

      list(
        mean  = as.numeric(fc_i$mean),
        lower = as.numeric(as.matrix(fc_i$lower)[, 1]),
        upper = as.numeric(as.matrix(fc_i$upper)[, 1]),
        ok = TRUE, error = NA_character_
      )
    }, error = function(e) {
      list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
           ok = FALSE, error = conditionMessage(e))
    })

  }, future.seed = TRUE)
})

plan(sequential)

fc_sarima_full       <- do.call(rbind, lapply(results_sarima, `[[`, "mean"))
fc_sarima_full_lower <- do.call(rbind, lapply(results_sarima, `[[`, "lower"))
fc_sarima_full_upper <- do.call(rbind, lapply(results_sarima, `[[`, "upper"))

n_failed <- sum(!sapply(results_sarima, `[[`, "ok"))
if (n_failed > 0) {
  cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
} else {
  cat("All", FORECAST_ORIGINS, "origins fit successfully.\n")
}

cat("SARIMA loop time:", round(time_sarima_full["elapsed"] / 60, 2), "minutes\n")

save(fc_sarima_full, fc_sarima_full_lower, fc_sarima_full_upper,
```

```
       file = sarima_full_path)
  cat("SARIMA forecasts + 95% PI saved.\n")
}
```

```
[cache hit] loading fc_sarima_full.RData - skipping SARIMA expanding window loop
```

```
mae_sarima_full <- sapply(1:h, function(hh) {
  mean(abs(fc_sarima_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh]), na.rm = TRUE)
})
rmse_sarima_full <- sapply(1:h, function(hh) {
  sqrt(mean((fc_sarima_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh])^2, na.rm = TR
})
cat("SARIMA MAE: ", round(mean(mae_sarima_full), 3), "\n")
```

```
SARIMA MAE:  109.129
```

```
cat("SARIMA RMSE:", round(mean(rmse_sarima_full), 3), "\n")
```

```
SARIMA RMSE: 139.602
```

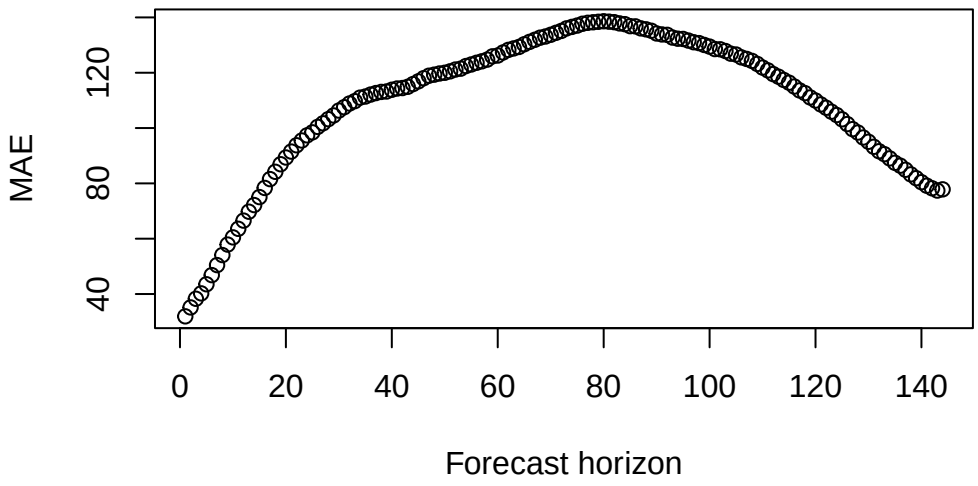
```
sarima_pi <- coverage_width(actual_matrix_full[1:FORECAST_ORIGINS, ], fc_sarima_full_lower, f
cat("SARIMA 95% coverage:", round(sarima_pi$coverage_overall, 3), " (target 0.95)\n")
```

```
SARIMA 95% coverage: 0.966  (target 0.95)
```

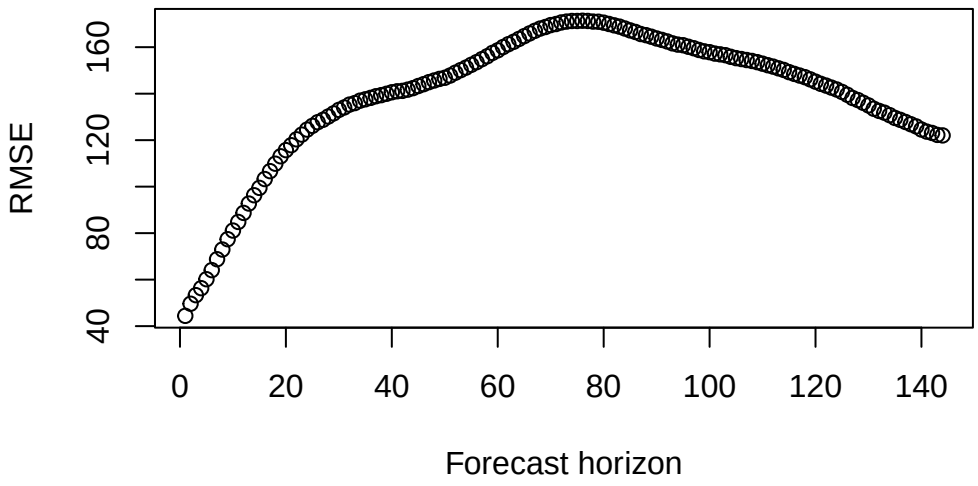
```
cat("SARIMA 95% width:   ", round(sarima_pi$width_overall, 2), "\n")
```

```
SARIMA 95% width:    699.98
```

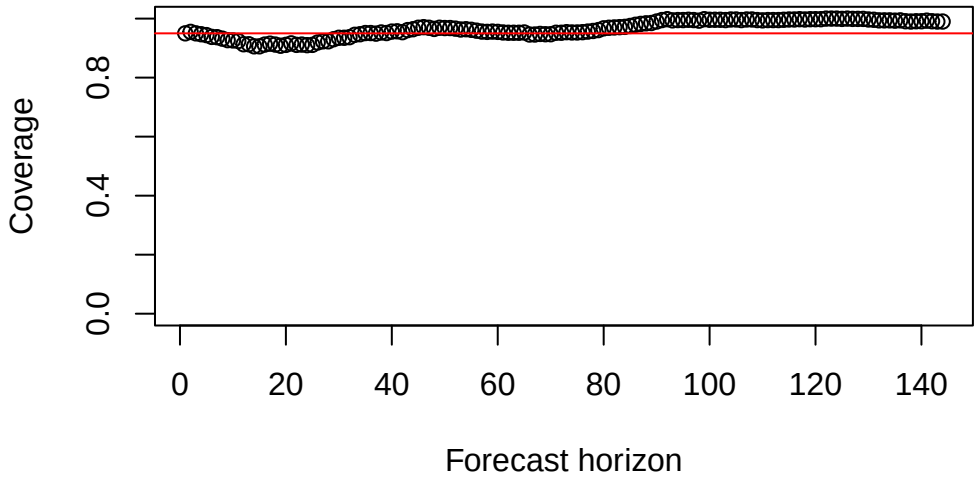
```
# SARIMA - MAE / RMSE by horizon
plot(mae_sarima_full, xlab = "Forecast horizon", ylab = "MAE")
```



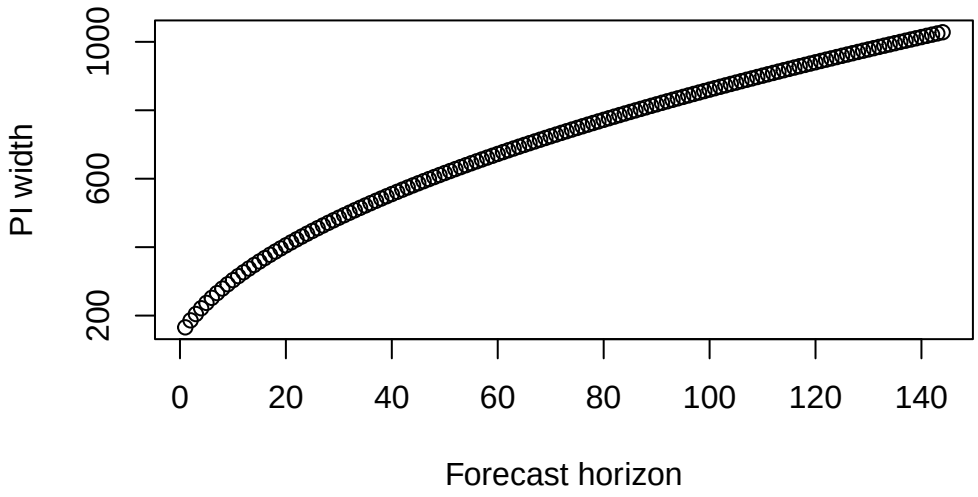

```
plot(rmse_sarima_full, xlab = "Forecast horizon", ylab = "RMSE")
```



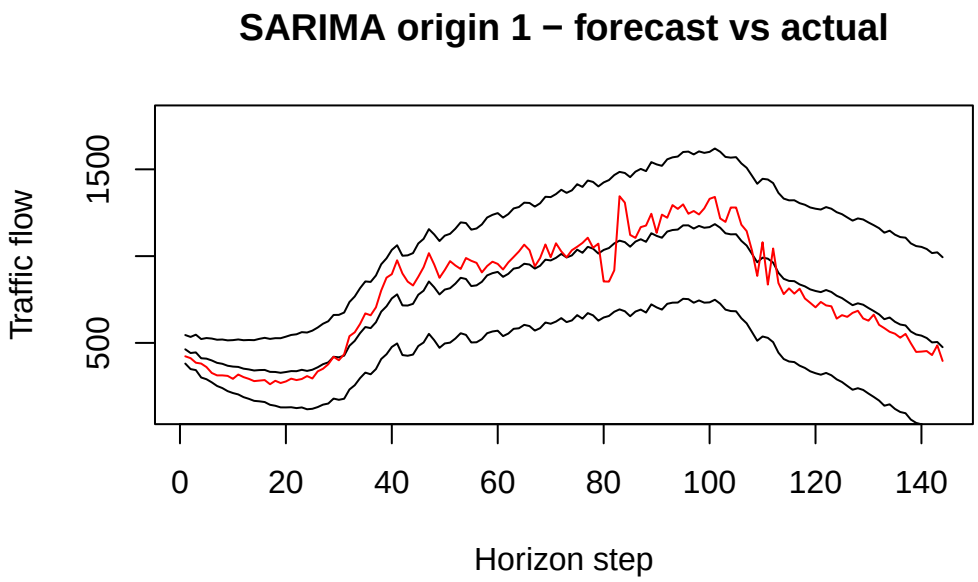
```r
# SARIMA - Coverage / width by horizon
plot(sarima_pi$coverage_by_h, ylim = c(0, 1), xlab = "Forecast horizon", ylab = "Coverage")
abline(h = 0.95, col = "red")
```



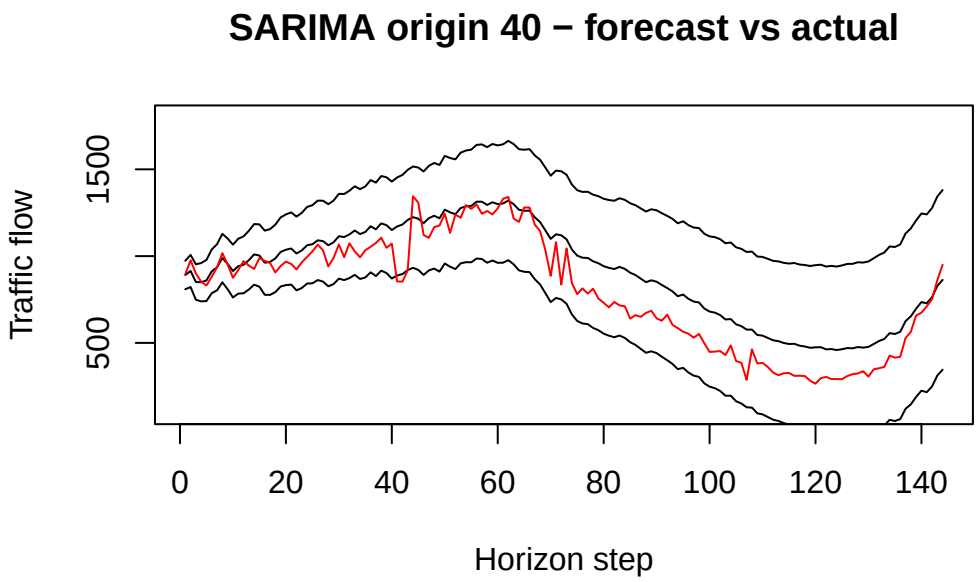

```r
plot(sarima_pi$width_by_h, xlab = "Forecast horizon", ylab = "PI width")
```




```
# SARIMA - representative forecasts, origins 1 and 40
plot(fc_sarima_full[1, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Tra
     main = "SARIMA origin 1 - forecast vs actual")
lines(fc_sarima_full_lower[1, ])
lines(fc_sarima_full_upper[1, ])
lines(actual_matrix_full[1, ], col = "red")
```




```
plot(fc_sarima_full[40, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Tr
     main = "SARIMA origin 40 - forecast vs actual")
lines(fc_sarima_full_lower[40, ])
lines(fc_sarima_full_upper[40, ])
lines(actual_matrix_full[40, ], col = "red")
```



TBATS

```
library(forecast)

y_train_msts <- msts(y_train, seasonal.periods = c(144,1008))

time_tbats <- system.time({
  fit_tbats <- tbats(y_train_msts,
           use.box.cox = NULL,
           use.trend = NULL,
           use.damped.trend = NULL,
           seasonal.periods = c(144,1008))
  })

cat("Fitting time:", time_tbats["elapsed"], "seconds\n")
```

```
Fitting time: 80.807 seconds
```

```r
print(fit_tbats)
```

```
TBATS(0.005, {4,4}, 0.801, {<144,8>, <1008,6>})

Call: tbats(y = y_train_msts, use.box.cox = NULL, use.trend = NULL,
    use.damped.trend = NULL, seasonal.periods = c(144, 1008))

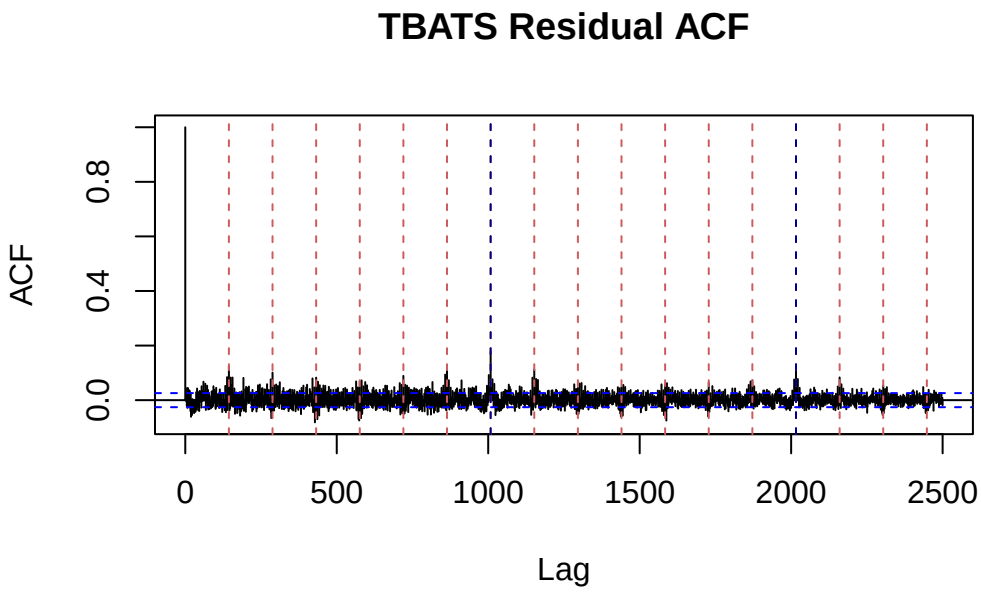
Parameters
  Lambda: 0.004785
  Alpha: 0.2495254
  Beta: 0.0648409
  Damping Parameter: 0.801164
  Gamma-1 Values: -4.770755e-05 -0.000225855
  Gamma-2 Values: -0.0002429958 -0.0006144312
  AR coefficients: 0.903199 -0.852945 -0.056227 -0.01304
  MA coefficients: -0.721984 0.696399 0.109367 0.041352

Seed States:
               [,1]
 [1,]  6.7888034240
 [2,] -0.0062923610
 [3,] -0.5100784268
 [4,] -0.0953758955
 [5,]  0.0818049254
 [6,]  0.0357640767
 [7,] -0.0275320842
 [8,] -0.0079935969
 [9,]  0.0061236444
[10,]  0.0064361197
[11,] -0.3108708764
[12,] -0.1219228432
[13,] -0.0644122583
[14,] -0.0005751635
[15,]  0.0227843194
[16,] -0.0013243475
[17,] -0.0137833505
[18,] -0.0030053320
[19,] -0.1390203952
[20,] -0.0955179873
[21,]  0.0038032108
[22,]  0.0343118887
[23,] -0.0297073598
```

```
[24,] -0.0267316706
[25,] -0.2175333378
[26,] -0.0117148553
[27,]  0.0606886280
[28,] -0.0059616675
[29,] -0.0622088803
[30,]  0.0810640516
[31,]  0.0000000000
[32,]  0.0000000000
[33,]  0.0000000000
[34,]  0.0000000000
[35,]  0.0000000000
[36,]  0.0000000000
[37,]  0.0000000000
[38,]  0.0000000000
attr(,"lambda")
[1] 0.004785086

Sigma: 0.05811207
AIC: 90919.05
```

```r
# Check TBATS residuals
acf(as.numeric(residuals(fit_tbats)), lag.max = 2500,
    main = "TBATS Residual ACF")
abline(v = seq(144,  2500, by = 144),  col = "indianred",  lty = 2)
abline(v = seq(1008, 2500, by = 1008), col = "darkblue", lty = 2)
```




```
# manually computing  MAE as accuracy(fit_tbats) returned NA :
fitted_tbats <- fitted(fit_tbats)
mae_tbats <- mean(abs(y_train - fitted_tbats), na.rm = TRUE)
rmse_tbats <- sqrt(mean((y_train - fitted_tbats)^2, na.rm = TRUE))
cat("TBATS MAE: ", mae_tbats, "\n")
```

```
TBATS MAE:  28.33635
```

```
cat("TBATS RMSE:", rmse_tbats, "\n")
```

```
TBATS RMSE: 41.37596
```

```
FORECAST_ORIGINS_tbats <- 1000
```

```
tbats_full_path <- "fc_tbats_full.RData"

if (file.exists(tbats_full_path)) {

  message("[cache hit] loading ", tbats_full_path, " - skipping TBATS expanding window loop")
  load(tbats_full_path)
```

```
} else {

  #n_workers <- max(1, parallel::detectCores() - 1)
  n_workers <- 4
  cat("Running TBATS expanding window (", FORECAST_ORIGINS_tbats, "refits) across", n_workers

  plan(multisession, workers = n_workers)

  time_tbats_full <- system.time({
    results_tbats <- future_lapply(1:FORECAST_ORIGINS_tbats, function(i) {

      train_end_i    <- n_train + i - 1
      y_train_i      <- y_full[1:train_end_i]
      y_train_msts_i <- forecast::msts(y_train_i, seasonal.periods = c(144, 1008))

      tryCatch({
        fit_i <- forecast::tbats(
          y_train_msts_i,
          use.box.cox      = TRUE,
          use.trend        = TRUE,
          use.damped.trend = TRUE,
          seasonal.periods  = c(144, 1008)
        )

        fc_i <- forecast::forecast(fit_i, h = h, level = 95)

        list(
          mean  = as.numeric(fc_i$mean),
          lower = as.numeric(as.matrix(fc_i$lower)[, 1]),
          upper = as.numeric(as.matrix(fc_i$upper)[, 1]),
          ok = TRUE, error = NA_character_
        )
      }, error = function(e) {
        list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
             ok = FALSE, error = conditionMessage(e))
      })

    }, future.seed = TRUE)
  })

  plan(sequential)
```

```
  fc_tbats_full       <- do.call(rbind, lapply(results_tbats, `[[`, "mean"))
  fc_tbats_full_lower <- do.call(rbind, lapply(results_tbats, `[[`, "lower"))
  fc_tbats_full_upper <- do.call(rbind, lapply(results_tbats, `[[`, "upper"))

  n_failed <- sum(!sapply(results_tbats, `[[`, "ok"))
  if (n_failed > 0) {
    cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
  } else {
    cat("All", FORECAST_ORIGINS_tbats, "origins fit successfully.\n")
  }

  cat("TBATS loop time:", round(time_tbats_full["elapsed"] / 60, 2), "minutes\n")

  save(fc_tbats_full, fc_tbats_full_lower, fc_tbats_full_upper, file = tbats_full_path)
  cat("TBATS forecasts + 95% PI saved.\n")
}
```

```
[cache hit] loading fc_tbats_full.RData - skipping TBATS expanding window loop
```

```
mae_tbats_full <- sapply(1:h, function(hh) {
  mean(abs(fc_tbats_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS_tbats, hh]), na.rm = T
})
rmse_tbats_full <- sapply(1:h, function(hh) {
  sqrt(mean((fc_tbats_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS_tbats, hh])^2, na.rm
})
cat("TBATS MAE: ", round(mean(mae_tbats_full), 3), "\n")
```

```
TBATS MAE:  105.292
```

```
cat("TBATS RMSE:", round(mean(rmse_tbats_full), 3), "\n")
```

```
TBATS RMSE: 144.791
```

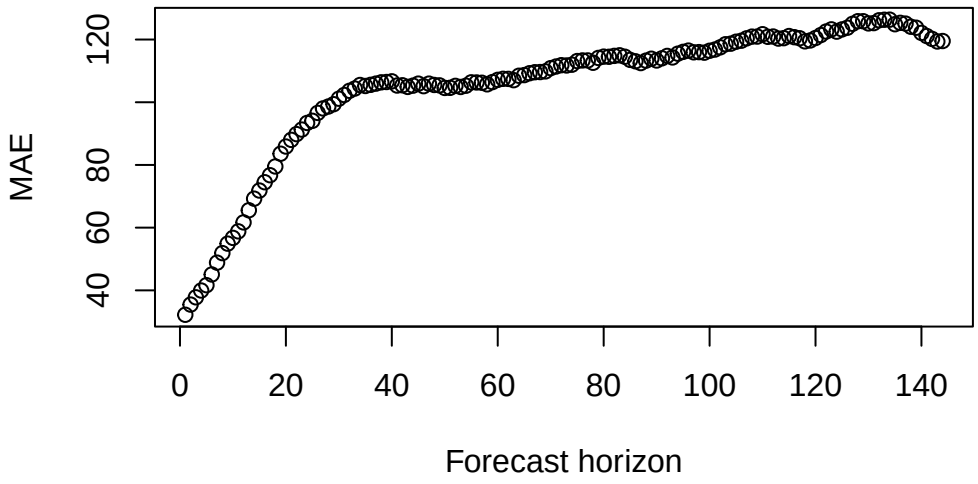
```
tbats_pi <- coverage_width(actual_matrix_full[1:FORECAST_ORIGINS_tbats, ], fc_tbats_full_lowe
cat("TBATS 95% coverage:", round(tbats_pi$coverage_overall, 3), " (target 0.95)\n")
```

```
TBATS 95% coverage: 0.866  (target 0.95)
```

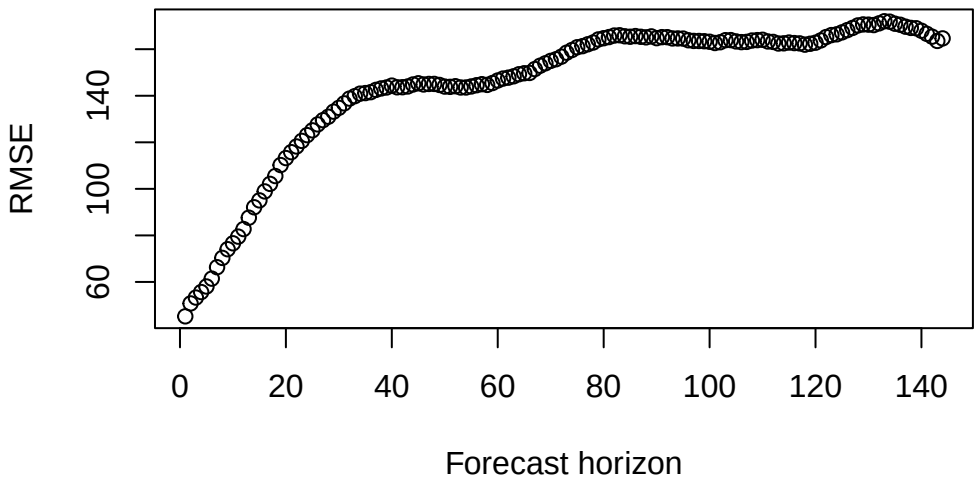
```r
cat("TBATS 95% width:   ", round(tbats_pi$width_overall, 2), "\n")
```

```
TBATS 95% width:    536.74
```

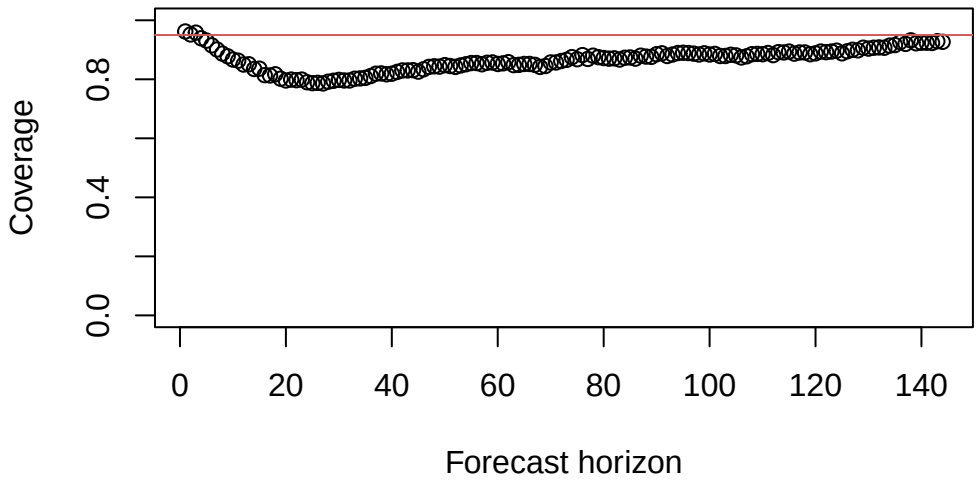
```r
plot(mae_tbats_full, xlab = "Forecast horizon", ylab = "MAE")
```



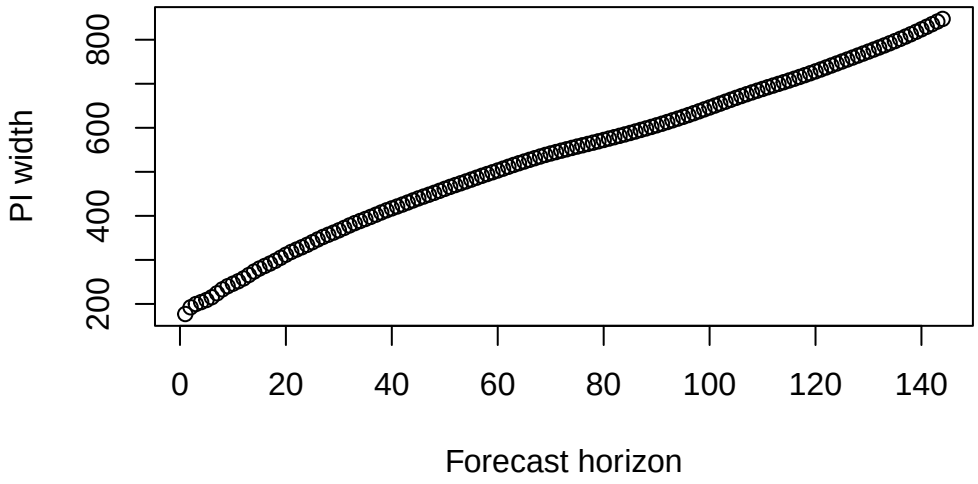

```r
plot(rmse_tbats_full, xlab = "Forecast horizon", ylab = "RMSE")
```



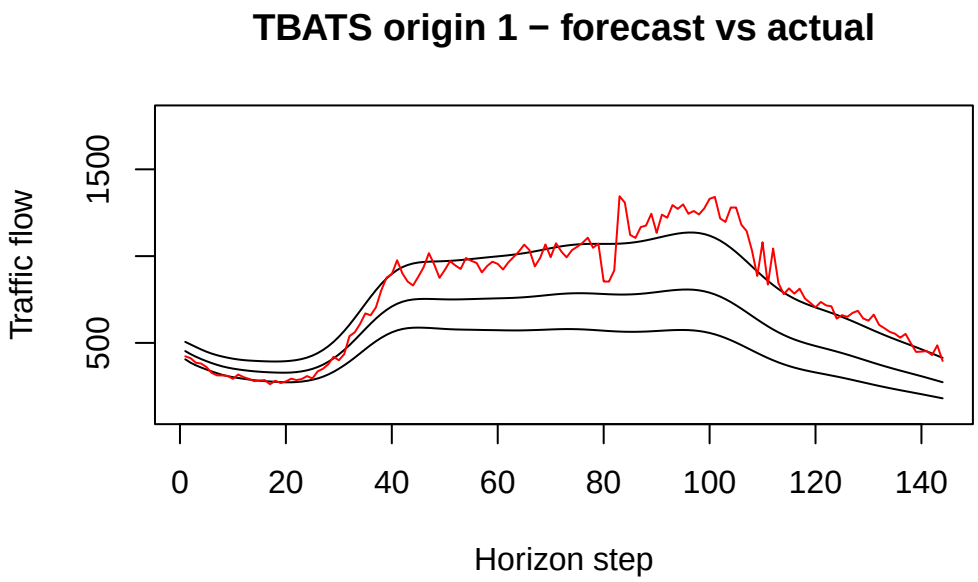

```
plot(tbats_pi$coverage_by_h, ylim = c(0, 1), xlab = "Forecast horizon", ylab = "Coverage")
abline(h = 0.95, col = "indianred")
```



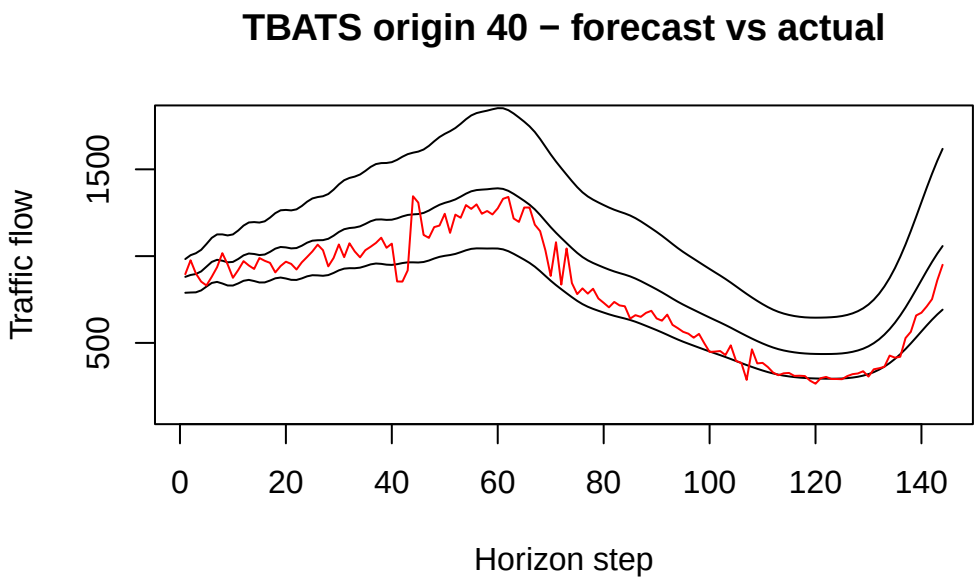
```r
plot(tbats_pi$width_by_h, xlab = "Forecast horizon", ylab = "PI width")
```




```r
# TBATS - representative forecasts, origins 1 and 40
plot(fc_tbats_full[1, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Tra
     main = "TBATS origin 1 - forecast vs actual")
lines(fc_tbats_full_lower[1, ])
lines(fc_tbats_full_upper[1, ])
lines(actual_matrix_full[1, ], col = "red")
```




```
plot(fc_tbats_full[40, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Tra
     main = "TBATS origin 40 - forecast vs actual")
lines(fc_tbats_full_lower[40, ])
lines(fc_tbats_full_upper[40, ])
lines(actual_matrix_full[40, ], col = "red")
```



MSTL

```
library(forecast)

y_train_msts <- msts(
  y_train,
  seasonal.periods = c(144, 1008)
)


# tune s.window, following Lisi & Grigoletto (2025):
# set the s.window value and other options to make the model work at its best, that is, minir


s_window_candidates <- list(
  periodic = "periodic",
  default  = c(11, 15),                    # mstl()'s own default for 2 periods
  w7       = 7,   w9  = 9,   w11 = 11,
  w13      = 13,  w15 = 15,  w17 = 17,
  w19      = 19,  w21 = 21
```

```
)

time_stage1 <- system.time({
  stage1_results <- lapply(names(s_window_candidates), function(nm) {
    sw  <- s_window_candidates[[nm]]
    fit <- stlf(y_train_msts, method = "ets", s.window = sw, robust = FALSE)

    fitted_vals <- fitted(fit)
    mae  <- mean(abs(y_train - fitted_vals), na.rm = TRUE)
    rmse <- sqrt(mean((y_train - fitted_vals)^2, na.rm = TRUE))

    cat(sprintf("s.window = %-10s MAE: %.5f  RMSE: %.5f\n", nm, mae, rmse))
    list(name = nm, s.window = sw, mae = mae, rmse = rmse, fit = fit)
  })
})
```

```
s.window = periodic   MAE: 23.11055  RMSE: 34.32456
s.window = default    MAE: 19.92514  RMSE: 29.59190
s.window = w7         MAE: 16.94242  RMSE: 25.04143
s.window = w9         MAE: 18.54827  RMSE: 27.54873
s.window = w11        MAE: 19.43264  RMSE: 28.89829
s.window = w13        MAE: 20.06882  RMSE: 29.87185
s.window = w15        MAE: 20.49840  RMSE: 30.54351
s.window = w17        MAE: 20.82932  RMSE: 31.05972
s.window = w19        MAE: 21.10833  RMSE: 31.50851
s.window = w21        MAE: 21.33804  RMSE: 31.89469
```

```
cat("Stage 1 tuning time:", time_stage1["elapsed"], "seconds\n")
```

```
Stage 1 tuning time: 1.995 seconds
```

```
best1    <- stage1_results[[which.min(sapply(stage1_results, `[[`, "mae"))]]
best_sw  <- best1$s.window
cat("\nStage 1 winner: s.window =", best1$name, " MAE =", round(best1$mae, 5), "\n")
```

```
Stage 1 winner: s.window = w7  MAE = 16.94242
```

```r
# ------------------------------------------------------------------
# Stage 2: with the winning s.window fixed, tune the "other option"
# robust loess fitting (downweights outliers in the smoothing step)
# ------------------------------------------------------------------

robust_candidates <- list(robust_off = FALSE, robust_on = TRUE)

time_stage2 <- system.time({
  stage2_results <- lapply(names(robust_candidates), function(nm) {
    rb  <- robust_candidates[[nm]]
    fit <- stlf(y_train_msts, method = "ets", s.window = best_sw, robust = rb)

    fitted_vals <- fitted(fit)
    mae  <- mean(abs(y_train - fitted_vals), na.rm = TRUE)
    rmse <- sqrt(mean((y_train - fitted_vals)^2, na.rm = TRUE))

    cat(sprintf("robust = %-10s MAE: %.5f  RMSE: %.5f\n", nm, mae, rmse))
    list(name = nm, robust = rb, mae = mae, rmse = rmse, fit = fit)
  })
})
```

```
robust = robust_off MAE: 16.94242  RMSE: 25.04143
robust = robust_on  MAE: 21.67648  RMSE: 39.56801
```

```r
cat("Stage 2 tuning time:", time_stage2["elapsed"], "seconds\n")
```

```
Stage 2 tuning time: 0.373 seconds
```

```r
best2 <- stage2_results[[which.min(sapply(stage2_results, `[[`, "mae"))]]
cat("\nStage 2 winner: robust =", best2$name, " MAE =", round(best2$mae, 5), "\n")
```

```
Stage 2 winner: robust = robust_off  MAE = 16.94242
```

```r
# ------------------------------------------------------------------
# Final model: winning s.window + winning robust setting
# ------------------------------------------------------------------

fit_mstl_fc <- best2$fit
fit_mstl    <- mstl(y_train_msts, s.window = best_sw, robust = best2$robust)
```

```r
fitted_mstl <- fitted(fit_mstl_fc)
mae_mstl  <- mean(abs(y_train - fitted_mstl), na.rm = TRUE)
rmse_mstl <- sqrt(mean((y_train - fitted_mstl)^2, na.rm = TRUE))

cat("\nFinal MSTL specification: s.window =", best1$name,
    " robust =", best2$name, "\n")
```

```
Final MSTL specification: s.window = w7  robust = robust_off
```

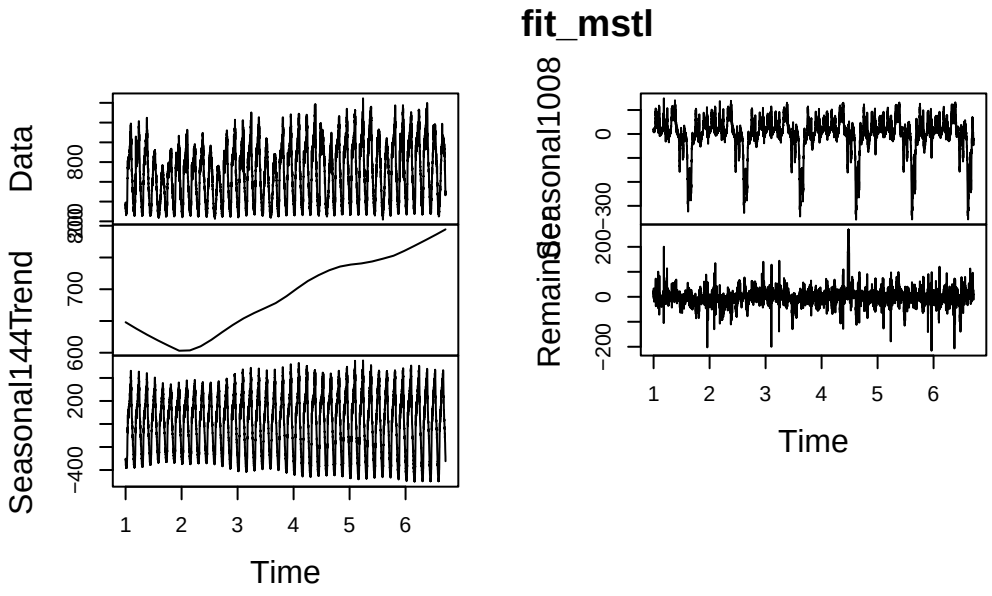
```r
cat("Final MAE: ", mae_mstl,  "\n")
```

```
Final MAE:  16.94242
```

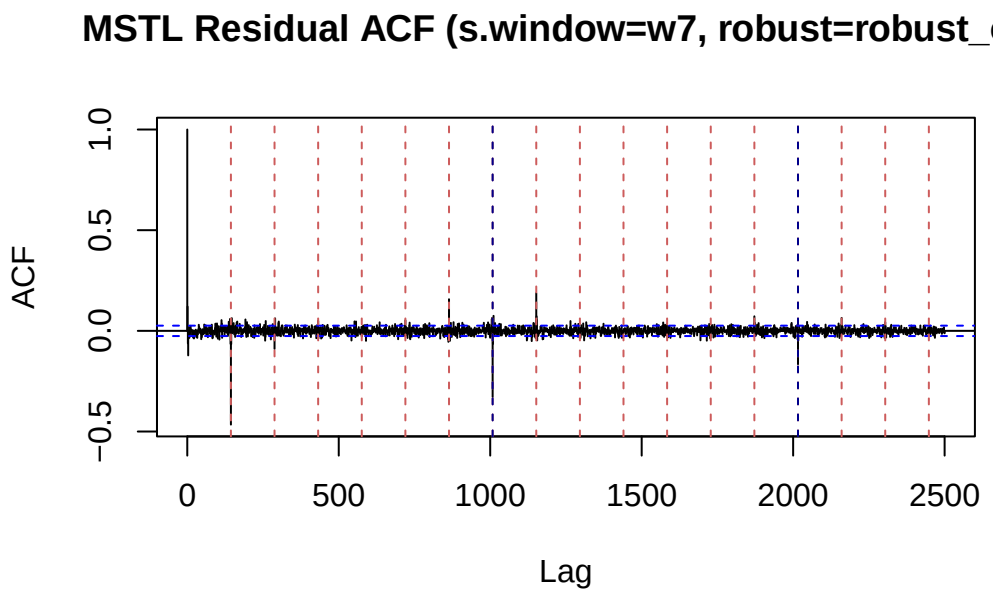
```r
cat("Final RMSE:", rmse_mstl, "\n")
```

```
Final RMSE: 25.04143
```

```r
# Plot decomposition for the selected specification
plot(fit_mstl)
```



```r
# Residual ACF
acf(as.numeric(residuals(fit_mstl_fc)), lag.max = 2500,
    main = paste0("MSTL Residual ACF (s.window=", best1$name,
                  ", robust=", best2$name, ")"))
abline(v = seq(144,  2500, by = 144),  col = "indianred", lty = 2)
abline(v = seq(1008, 2500, by = 1008), col = "darkblue",  lty = 2)
```




```r
## Forecast using MSTL decomposition (selected s.window)
# In-sample MAE for the selected model
fitted_mstl <- fitted(fit_mstl_fc)
mae_mstl  <- mean(abs(y_train - fitted_mstl), na.rm = TRUE)
rmse_mstl <- sqrt(mean((y_train - fitted_mstl)^2, na.rm = TRUE))

cat("Selected s.window:", best_sw, "\n")
```

```
Selected s.window: 7
```

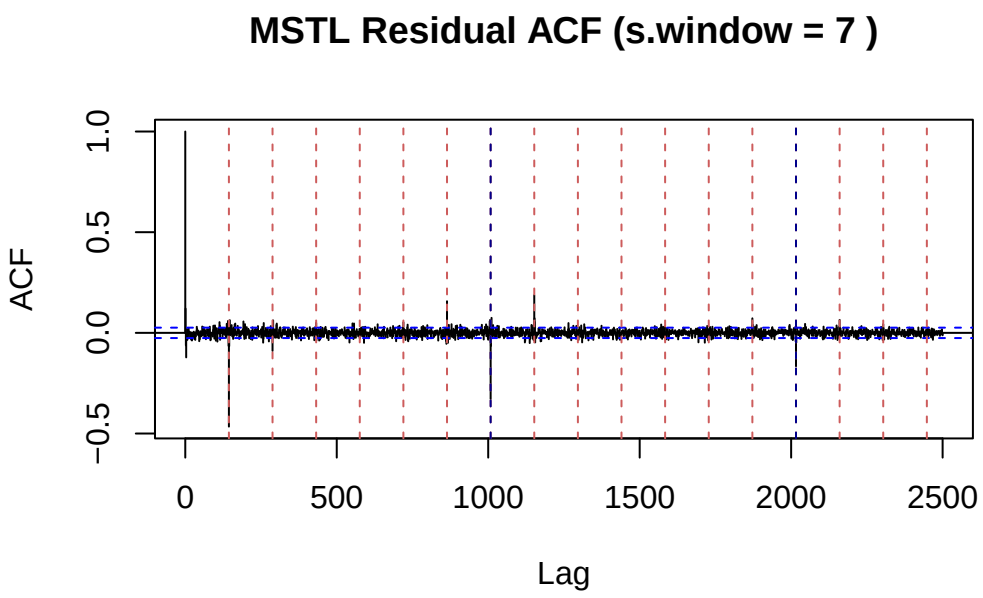
```r
cat("MAE: ", mae_mstl,  "\n")
```

```
MAE:  16.94242
```

```r
cat("RMSE:", rmse_mstl, "\n")
```

```
RMSE: 25.04143
```

```r
# Residual ACF
acf(as.numeric(residuals(fit_mstl_fc)), lag.max = 2500,
    main = paste("MSTL Residual ACF (s.window =", best_sw, ")"))
abline(v = seq(144,  2500, by = 144),  col = "indianred", lty = 2)
abline(v = seq(1008, 2500, by = 1008), col = "darkblue",  lty = 2)
```




```r
# stlf()'s level is a PERCENTAGE (0-100) - level = 95, not 0.95. Different
# convention than mSARIMA/SARIMA above (smooth uses 0-1 proportions).
library(future.apply)
library(future)


mstl_full_path <- "fc_mstl_full.RData"

if (file.exists(mstl_full_path)) {

  message("[cache hit] loading ", mstl_full_path, " - skipping MSTL expanding window loop")
```

```
  load(mstl_full_path)

} else {

  n_workers <- max(1, parallel::detectCores() - 1)
  cat("Running MSTL expanding window (", FORECAST_ORIGINS, "refits) across",
      n_workers, "workers...\n")

  plan(multisession, workers = n_workers)

  time_mstl_full <- system.time({
    results_mstl <- future_lapply(1:FORECAST_ORIGINS, function(i) {

      train_end_i    <- n_train + i - 1
      y_train_i      <- y_full[1:train_end_i]
      y_train_msts_i <- forecast::msts(y_train_i, seasonal.periods = c(144, 1008))

      tryCatch({
        fc_i <- forecast::stlf(
          y_train_msts_i,
          h        = h,
          method   = "ets",
          s.window = best_sw,       # winning setting from your Stage 1 tuning (w7 = 7)
          robust   = best2$robust,  # winning setting from your Stage 2 tuning (robust_off =
          level    = 95
        )

        list(
          mean  = as.numeric(fc_i$mean),
          lower = as.numeric(as.matrix(fc_i$lower)[, 1]),
          upper = as.numeric(as.matrix(fc_i$upper)[, 1]),
          ok = TRUE, error = NA_character_
        )
      }, error = function(e) {
        list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
             ok = FALSE, error = conditionMessage(e))
      })

    }, future.seed = TRUE)
  })

  plan(sequential)
```

```
  fc_mstl_full       <- do.call(rbind, lapply(results_mstl, `[[`, "mean"))
  fc_mstl_full_lower <- do.call(rbind, lapply(results_mstl, `[[`, "lower"))
  fc_mstl_full_upper <- do.call(rbind, lapply(results_mstl, `[[`, "upper"))

  n_failed <- sum(!sapply(results_mstl, `[[`, "ok"))
  if (n_failed > 0) {
    cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
  } else {
    cat("All", FORECAST_ORIGINS, "origins fit successfully.\n")
  }

  cat("MSTL loop time:", round(time_mstl_full["elapsed"]/60, 2), "minutes\n")

  save(fc_mstl_full, fc_mstl_full_lower, fc_mstl_full_upper, file = mstl_full_path)
  cat("MSTL forecasts + 95% PI saved.\n")
}
```

```
[cache hit] loading fc_mstl_full.RData - skipping MSTL expanding window loop
```

```
mae_mstl_full <- sapply(1:h, function(hh) {
  mean(abs(fc_mstl_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh]), na.rm = TRUE)
})
rmse_mstl_full <- sapply(1:h, function(hh) {
  sqrt(mean((fc_mstl_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh])^2, na.rm = TRU
})
cat("MSTL MAE: ", round(mean(mae_mstl_full), 3), "\n")
```

```
MSTL MAE:  57.495
```

```
cat("MSTL RMSE:", round(mean(rmse_mstl_full), 3), "\n")
```

```
MSTL RMSE: 73.96
```

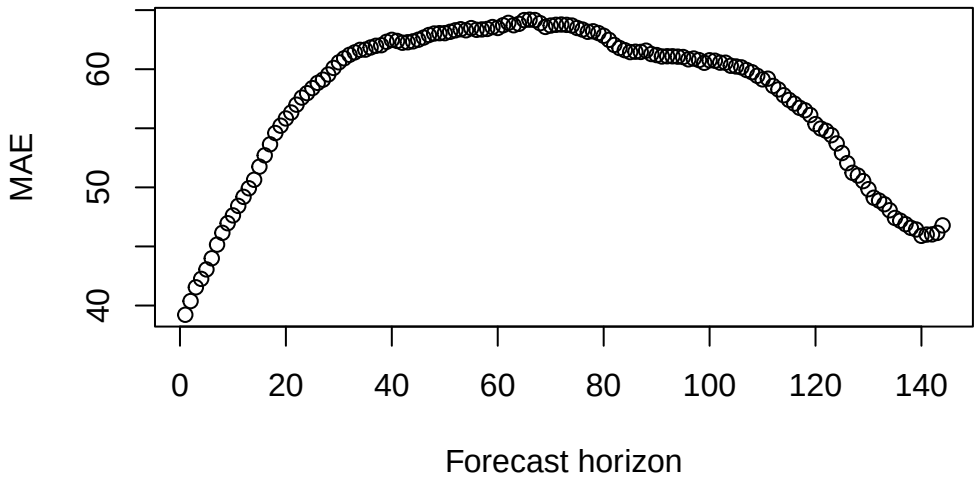
```
mstl_pi <- coverage_width(actual_matrix_full[1:FORECAST_ORIGINS, ], fc_mstl_full_lower, fc_ms
cat("MSTL 95% coverage:", round(mstl_pi$coverage_overall, 3), " (target 0.95)\n")
```

```
MSTL 95% coverage: 0.886  (target 0.95)
```

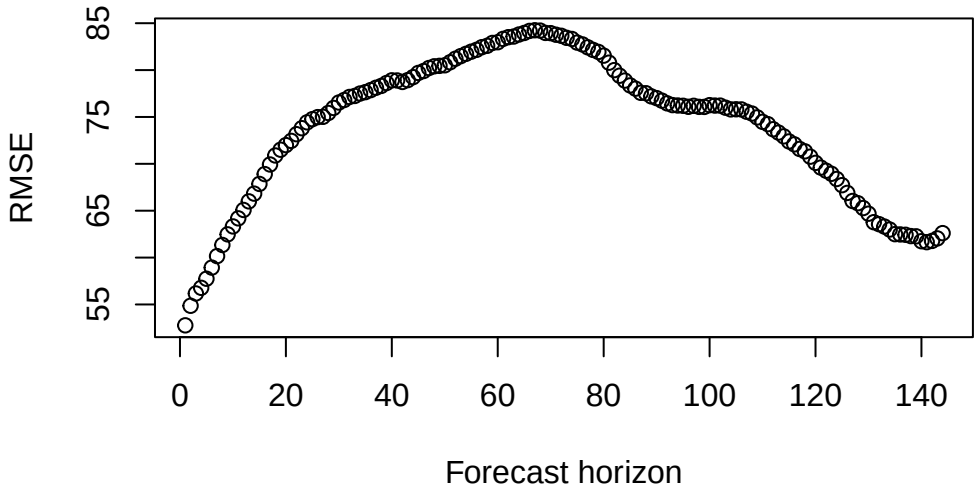
```r
cat("MSTL 95% width:   ", round(mstl_pi$width_overall, 2), "\n")
```

```
MSTL 95% width:    258.26
```

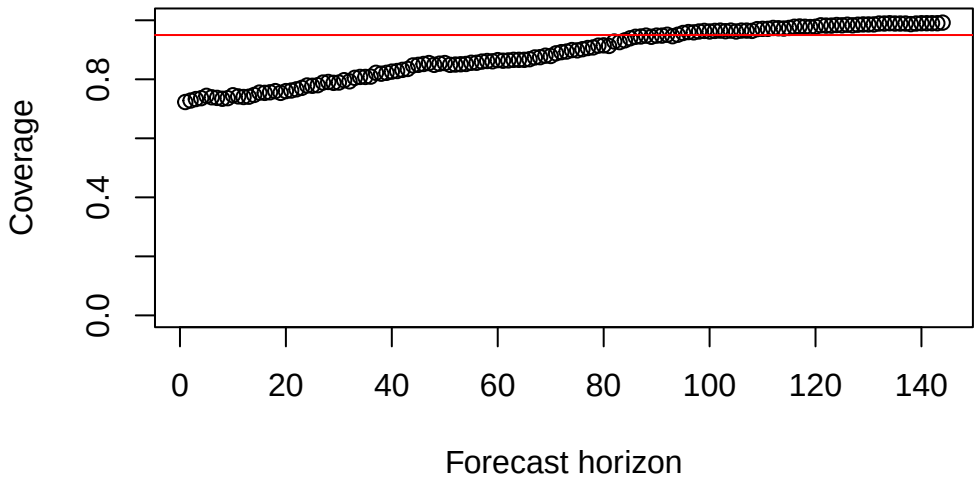
```r
plot(mae_mstl_full, xlab = "Forecast horizon", ylab = "MAE")
```



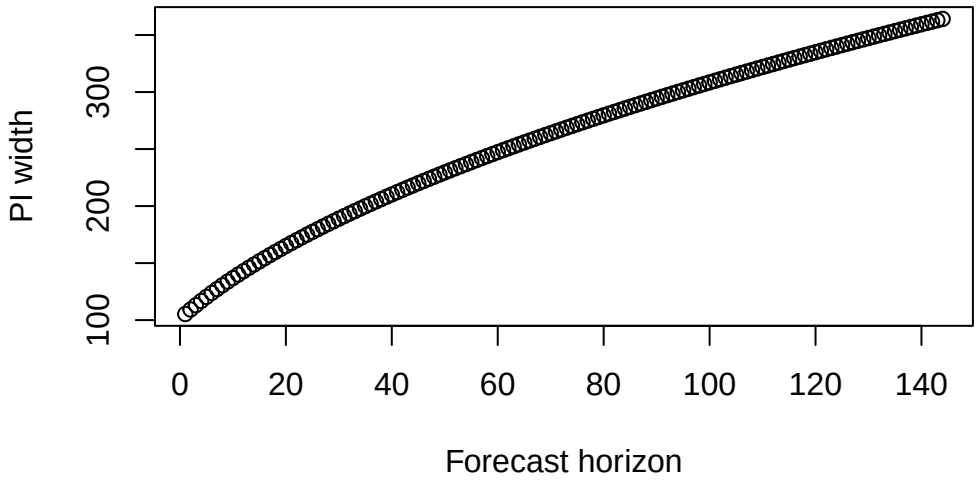

```r
plot(rmse_mstl_full, xlab = "Forecast horizon", ylab = "RMSE")
```



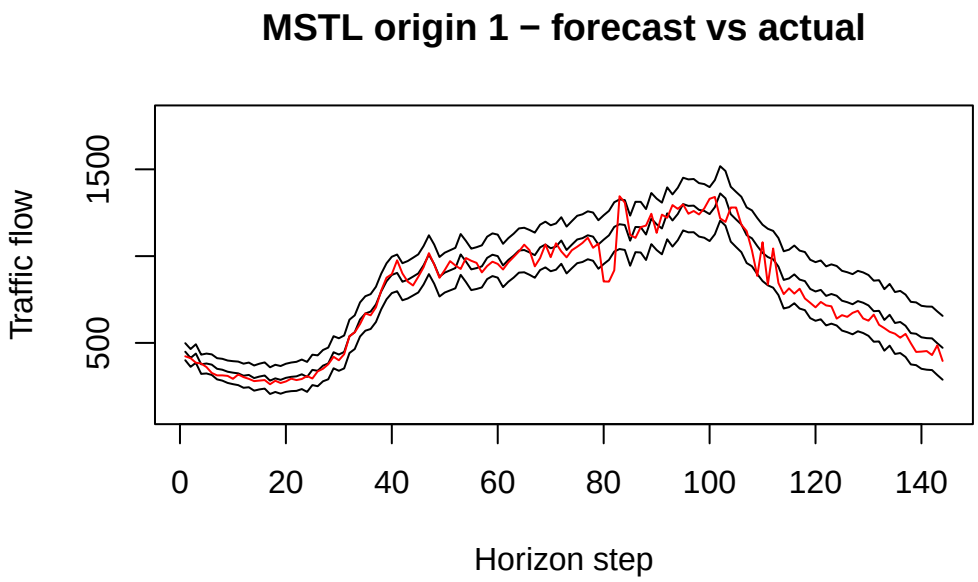

```
plot(mstl_pi$coverage_by_h, ylim = c(0, 1), xlab = "Forecast horizon", ylab = "Coverage")
abline(h = 0.95, col = "red")
```



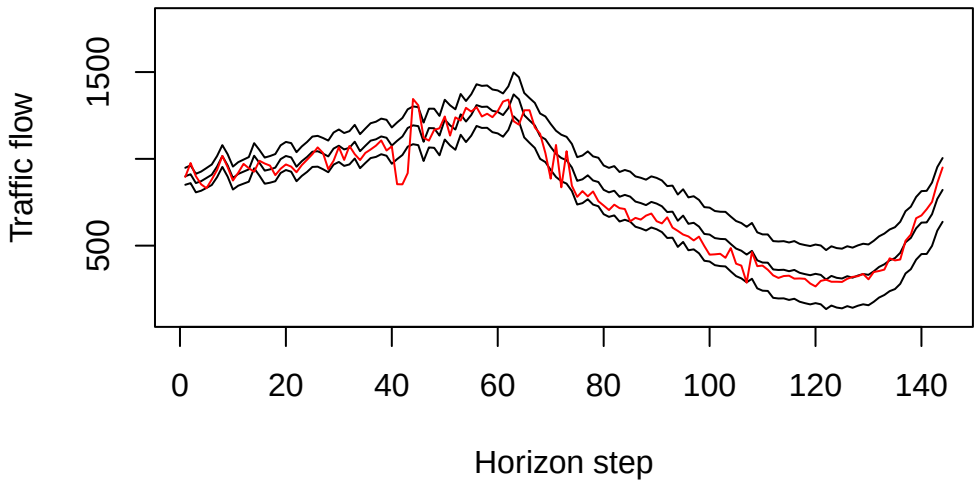
```r
plot(mstl_pi$width_by_h, xlab = "Forecast horizon", ylab = "PI width")
```




```r
# MSTL
plot(fc_mstl_full[1, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Traf
     main = "MSTL origin 1 - forecast vs actual")
lines(fc_mstl_full_lower[1, ]); lines(fc_mstl_full_upper[1, ]); lines(actual_matrix_full[1, ]
```




```
plot(fc_mstl_full[40, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Tra
     main = "MSTL origin 40 - forecast vs actual")
lines(fc_mstl_full_lower[40, ]); lines(fc_mstl_full_upper[40, ]); lines(actual_matrix_full[4
```



Prophet

```
# ============================================================
# Prophet: model identification (Fourier order + changepoint.prior.scale)
# and final model fit
# ============================================================

library(prophet)

prophet_train <- data.frame(
  ds = as.POSIXct(traffic_train$datetime, tz = "UTC"),
  y  = y_train
)
# ---- shared helper: build + fit a Prophet model for given settings ----
fit_prophet_model <- function(train_data, daily_k, weekly_k, cps) {
  m <- prophet(
    daily.seasonality  = FALSE,
    weekly.seasonality = FALSE,
    yearly.seasonality = FALSE,
    seasonality.mode   = "additive",
    changepoint.prior.scale = cps
  )
  m <- add_seasonality(m, name = "daily",  period = 1, fourier.order = daily_k)
  m <- add_seasonality(m, name = "weekly", period = 7, fourier.order = weekly_k)
  fit.prophet(m, train_data)
}

# ---- Stage 1: Fourier-order grid search (cps fixed at Prophet's default 0.05) ----
fourier_grid <- expand.grid(daily_order  = c(4, 6, 8, 10),
                            weekly_order = c(3, 5, 7, 10))

prophet_results <- lapply(seq_len(nrow(fourier_grid)), function(i) {
  daily_k  <- fourier_grid$daily_order[i]
  weekly_k <- fourier_grid$weekly_order[i]

  fit <- fit_prophet_model(prophet_train, daily_k, weekly_k, cps = 0.05)
  fc  <- predict(fit, prophet_train)
  mae <- mean(abs(prophet_train$y - fc$yhat))

  data.frame(daily_order = daily_k, weekly_order = weekly_k, MAE = mae)
})
```

```r
prophet_results <- do.call(rbind, prophet_results)
prophet_results <- prophet_results[order(prophet_results$MAE), ]
print(prophet_results)
```

```
   daily_order weekly_order      MAE
16          10           10 50.22610
15           8           10 50.23851
14           6           10 50.67188
12          10            7 51.94827
11           8            7 51.97387
10           6            7 52.40225
13           4           10 53.59344
9            4            7 55.48777
7            8            5 61.10535
8           10            5 61.12223
6            6            5 61.47565
5            4            5 63.89964
4           10            3 65.68429
3            8            3 65.69551
2            6            3 66.06765
1            4            3 68.55632
```

```r
# ---- Stage 1b: extend weekly_order at fixed daily_order = 10 ----
weekly_extend <- c(10, 13, 16, 20)

prophet_stage1b <- lapply(weekly_extend, function(weekly_k) {
  fit <- fit_prophet_model(prophet_train, daily_k = 10, weekly_k, cps = 0.05)
  fc  <- predict(fit, prophet_train)
  mae <- mean(abs(prophet_train$y - fc$yhat))
  data.frame(weekly_order = weekly_k, MAE = mae)
})
prophet_stage1b <- do.call(rbind, prophet_stage1b)
print(prophet_stage1b)
```

```
  weekly_order      MAE
1           10 50.22610
2           13 49.08318
3           16 47.17994
4           20 44.00634
```

```r
# ---- Stage 2: changepoint.prior.scale grid at fixed daily_k = 10, weekly_k = 20 ----
cps_grid <- c(0.001, 0.01, 0.05, 0.1, 0.5)

prophet_stage2 <- lapply(cps_grid, function(cps) {
  fit <- fit_prophet_model(prophet_train, daily_k = 10, weekly_k = 20, cps)
  fc  <- predict(fit, prophet_train)
  mae <- mean(abs(prophet_train$y - fc$yhat))
  data.frame(cps = cps, MAE = mae)
})
prophet_stage2 <- do.call(rbind, prophet_stage2)
print(prophet_stage2)
```

```
    cps      MAE
1 0.001 46.40094
2 0.010 44.62990
3 0.050 44.00634
4 0.100 43.75538
5 0.500 43.24012
```

```r
# ---- Final model: best settings from Stage 1/1b/2 ----
best_daily  <- 10
best_weekly <- 20
best_cps    <- 0.5

time_prophet_final <- system.time({
  fit_prophet <- fit_prophet_model(prophet_train, best_daily, best_weekly, best_cps)
})
cat("Final Prophet fitting time:", time_prophet_final["elapsed"], "seconds\n")
```

```
Final Prophet fitting time: 2.158 seconds
```

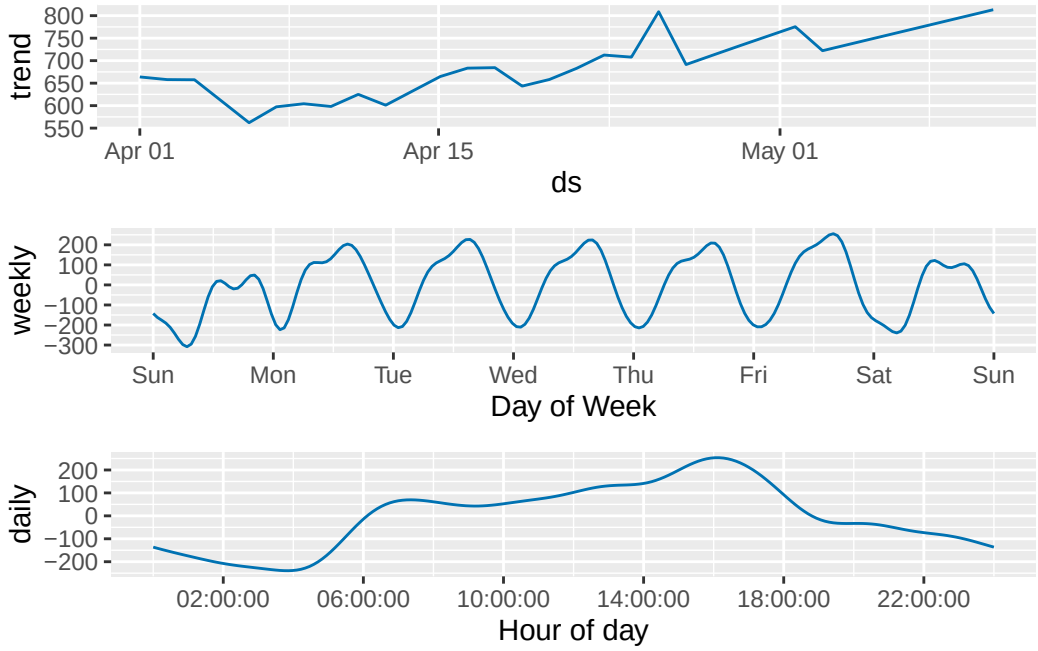
```r
prophet_fitted <- predict(fit_prophet, prophet_train)

mae  <- mean(abs(prophet_train$y - prophet_fitted$yhat))
rmse <- sqrt(mean((prophet_train$y - prophet_fitted$yhat)^2))
cat("Final Prophet MAE:", mae, " RMSE:", rmse, "\n")
```

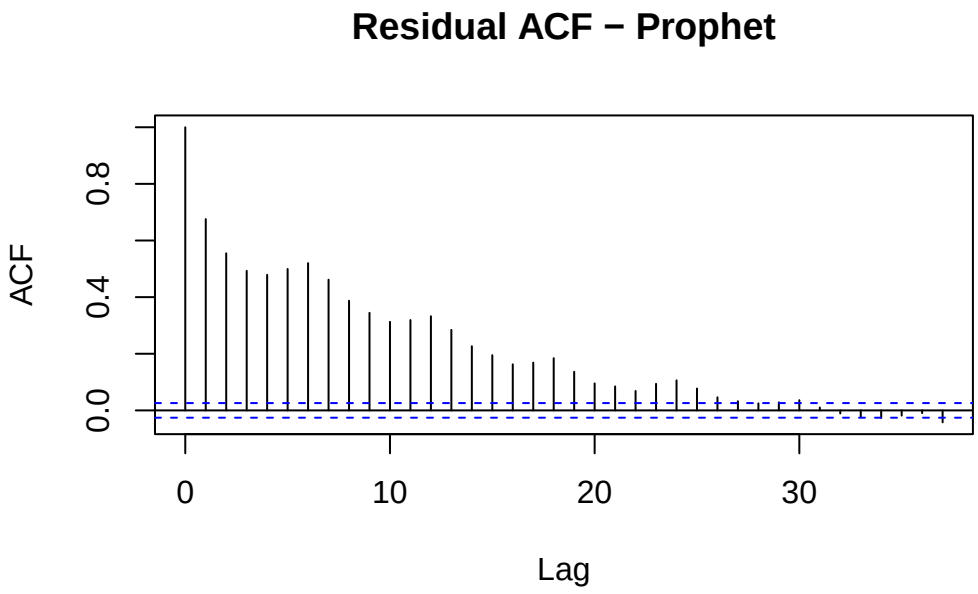
```
Final Prophet MAE: 43.24012  RMSE: 57.94717
```

```r
# ---- diagnostics ----
prophet_plot_components(fit_prophet, prophet_fitted)
```

```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the prophet package.
  Please report the issue at <https://github.com/facebook/prophet/issues>.
```




```r
acf(prophet_train$y - prophet_fitted$yhat, main = "Residual ACF - Prophet")
```



Expand window

```
#
# library(future.apply)
# library(future)
#
# prophet_full_path <- "fc_prophet_full.RData"
#
# if (file.exists(prophet_full_path)) {
#
#   message("[cache hit] loading ", prophet_full_path, " - skipping Prophet expanding window
#   load(prophet_full_path)
#
# } else {
#
#   n_workers <- max(1, parallel::detectCores() - 1)
#   cat("Running Prophet FULL expanding window (3024 refits) across",
#       n_workers, "workers...\n")
#   cat("Slowest of the six models - expect a long run even parallelized.\n")
#
```

```
#   plan(multisession, workers = n_workers)
#
#   time_prophet_full <- system.time({
#     results_prophet <- future_lapply(1:3024, function(i) {
#
#       train_end_i <- n_train + i - 1
#       y_train_i   <- y_full[1:train_end_i]
#
#       prophet_df <- data.frame(
#         ds = as.POSIXct(traffic_10min$datetime[1:train_end_i], tz = "UTC"),
#         y  = y_train_i
#       )
#
#       tryCatch({
#         fit_i <- suppressMessages(
#           prophet::prophet(
#             prophet_df,
#             daily.seasonality  = TRUE,
#             weekly.seasonality = TRUE,
#             yearly.seasonality = FALSE,
#             seasonality.mode   = "additive",
#             interval.width     = 0.95
#           )
#         )
#
#         future_df <- data.frame(
#           ds = as.POSIXct(
#             traffic_10min$datetime[(train_end_i + 1):(train_end_i + h)],
#             tz = "UTC"
#           )
#         )
#
#         fc_i <- predict(fit_i, future_df)
#
#         list(mean = fc_i$yhat, lower = fc_i$yhat_lower, upper = fc_i$yhat_upper,
#              ok = TRUE, error = NA_character_)
#       }, error = function(e) {
#         list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
#              ok = FALSE, error = conditionMessage(e))
#       })
#
#     }, future.seed = TRUE)
```

```
#   })
#
#   plan(sequential)
#
#   fc_prophet_full       <- do.call(rbind, lapply(results_prophet, `[[`, "mean"))
#   fc_prophet_full_lower <- do.call(rbind, lapply(results_prophet, `[[`, "lower"))
#   fc_prophet_full_upper <- do.call(rbind, lapply(results_prophet, `[[`, "upper"))
#
#   n_failed <- sum(!sapply(results_prophet, `[[`, "ok"))
#   if (n_failed > 0) {
#     cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
#     failed_idx <- which(!sapply(results_prophet, `[[`, "ok"))
#     cat("Failed origins:", paste(head(failed_idx, 10), collapse = ", "),
#         if (length(failed_idx) > 10) "..." else "", "\n")
#   } else {
#     cat("All 3024 origins fit successfully.\n")
#   }
#
#   cat("Prophet full loop time:",
#       round(time_prophet_full["elapsed"]/60, 2), "minutes\n")
#
#   save(fc_prophet_full, fc_prophet_full_lower, fc_prophet_full_upper,
#        file = prophet_full_path)
#   cat("Prophet full forecasts + 95% PI saved.\n")
# }
```

```
library(future.apply)
library(future)

prophet_full_path <- "fc_prophet_full.RData"

if (file.exists(prophet_full_path)) {

  message("[cache hit] loading ", prophet_full_path, " - skipping Prophet expanding window lo
  load(prophet_full_path)

} else {

  n_workers <- max(1, parallel::detectCores() - 1)
  cat("Running Prophet expanding window (", FORECAST_ORIGINS, "refits) across",
      n_workers, "workers...\n")
  cat("Likely one of the slower models - expect a longer run even parallelized.\n")
```

```r
plan(multisession, workers = n_workers)

time_prophet_full <- system.time({
  results_prophet <- future_lapply(1:FORECAST_ORIGINS, function(i) {

    train_end_i <- n_train + i - 1
    y_train_i   <- y_full[1:train_end_i]

    prophet_df <- data.frame(
      ds = as.POSIXct(traffic_10min$datetime[1:train_end_i], tz = "UTC"),
      y  = y_train_i
    )

    tryCatch({
      m_i <- prophet::prophet(
        daily.seasonality  = FALSE,
        weekly.seasonality = FALSE,
        yearly.seasonality = FALSE,
        seasonality.mode   = "additive",
        changepoint.prior.scale = 0.5,
        interval.width     = 0.95
      )
      m_i <- prophet::add_seasonality(m_i, name = "daily",  period = 1, fourier.order = 10)
      m_i <- prophet::add_seasonality(m_i, name = "weekly", period = 7, fourier.order = 20)
      fit_i <- suppressMessages(prophet::fit.prophet(m_i, prophet_df))

      future_df <- data.frame(
        ds = as.POSIXct(
          traffic_10min$datetime[(train_end_i + 1):(train_end_i + h)],
          tz = "UTC"
        )
      )

      fc_i <- predict(fit_i, future_df)

      list(mean = fc_i$yhat, lower = fc_i$yhat_lower, upper = fc_i$yhat_upper,
           ok = TRUE, error = NA_character_)
    }, error = function(e) {
      list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
           ok = FALSE, error = conditionMessage(e))
    })
```

```
    }, future.seed = TRUE)
  })

  plan(sequential)

  fc_prophet_full       <- do.call(rbind, lapply(results_prophet, `[[`, "mean"))
  fc_prophet_full_lower <- do.call(rbind, lapply(results_prophet, `[[`, "lower"))
  fc_prophet_full_upper <- do.call(rbind, lapply(results_prophet, `[[`, "upper"))

  n_failed <- sum(!sapply(results_prophet, `[[`, "ok"))
  if (n_failed > 0) {
    cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
    failed_idx <- which(!sapply(results_prophet, `[[`, "ok"))
    cat("Failed origins:", paste(head(failed_idx, 10), collapse = ", "),
        if (length(failed_idx) > 10) "..." else "", "\n")
  } else {
    cat("All", FORECAST_ORIGINS, "origins fit successfully.\n")
  }

  cat("Prophet loop time:", round(time_prophet_full["elapsed"]/60, 2), "minutes\n")

  save(fc_prophet_full, fc_prophet_full_lower, fc_prophet_full_upper, file = prophet_full_pat
  cat("Prophet forecasts + 95% PI saved.\n")
}
```

```
[cache hit] loading fc_prophet_full.RData - skipping Prophet expanding window loop
```

```
mae_prophet_full <- sapply(1:h, function(hh) {
  mean(abs(fc_prophet_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh]), na.rm = TRUE)
})
rmse_prophet_full <- sapply(1:h, function(hh) {
  sqrt(mean((fc_prophet_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh])^2, na.rm = T
})
cat("Prophet MAE: ", round(mean(mae_prophet_full), 3), "\n")
```

```
Prophet MAE:  64.73
```

```
cat("Prophet RMSE:", round(mean(rmse_prophet_full), 3), "\n")
```

```
Prophet RMSE: 78.857
```

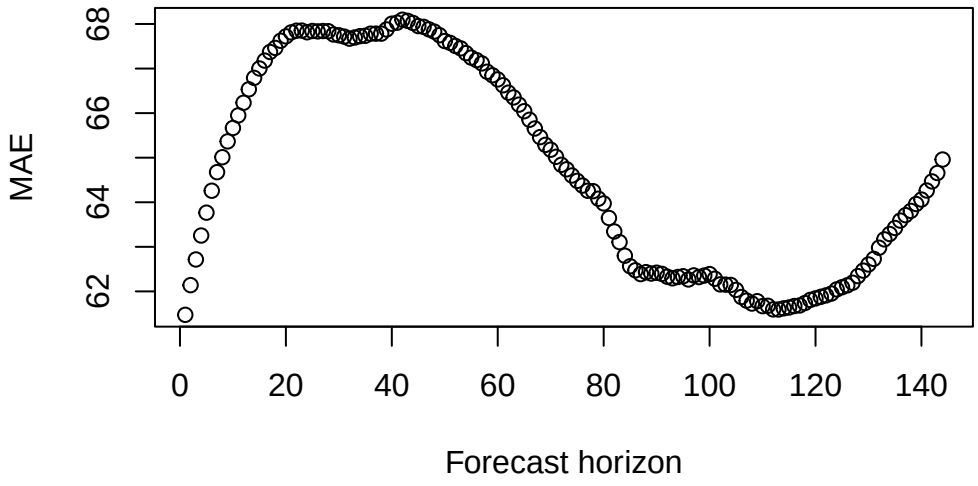
```r
prophet_pi <- coverage_width(actual_matrix_full[1:FORECAST_ORIGINS, ], fc_prophet_full_lower
cat("Prophet 95% coverage:", round(prophet_pi$coverage_overall, 3), " (target 0.95)\n")
```

```
Prophet 95% coverage: 0.889  (target 0.95)
```

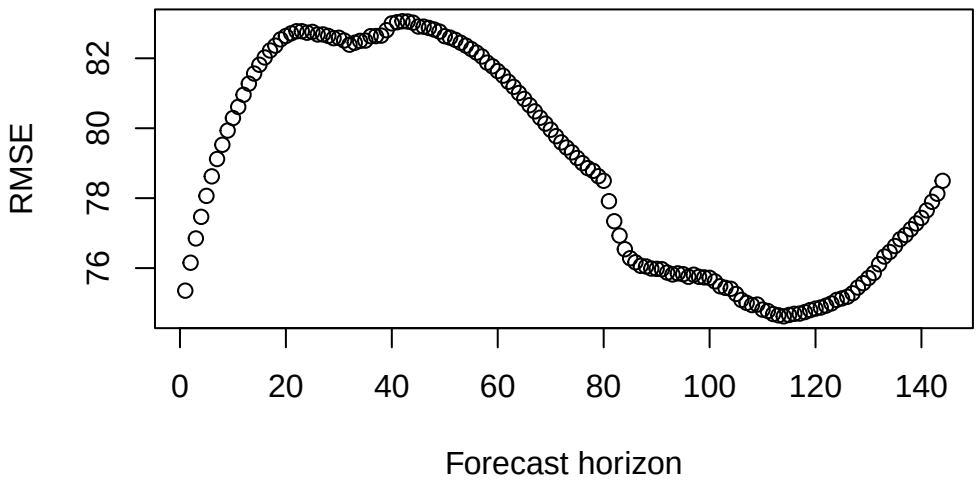
```r
cat("Prophet 95% width:   ", round(prophet_pi$width_overall, 2), "\n")
```

```
Prophet 95% width:    233.95
```

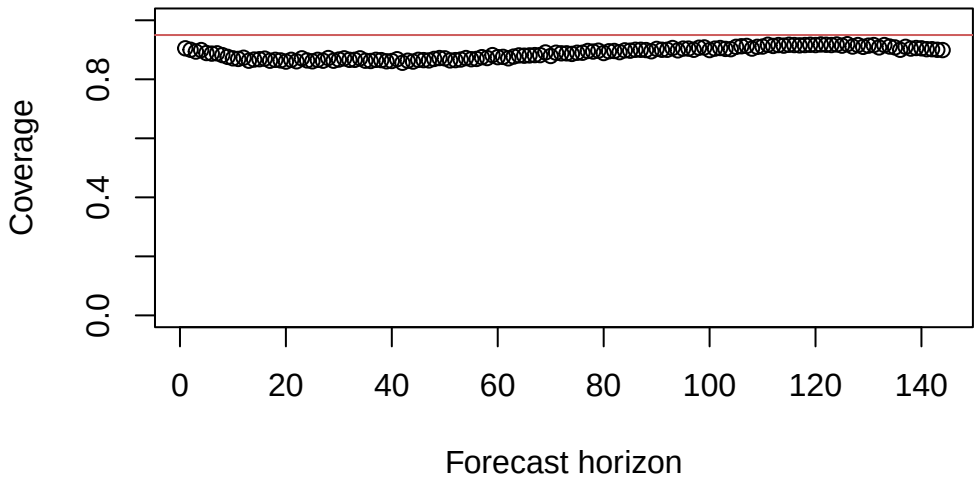
```r
plot(mae_prophet_full, xlab = "Forecast horizon", ylab = "MAE")
```



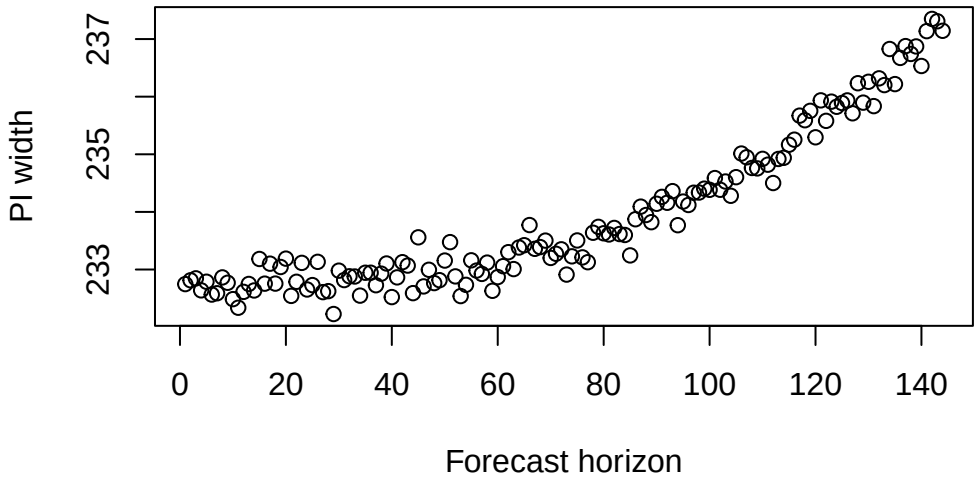

```r
plot(rmse_prophet_full, xlab = "Forecast horizon", ylab = "RMSE")
```



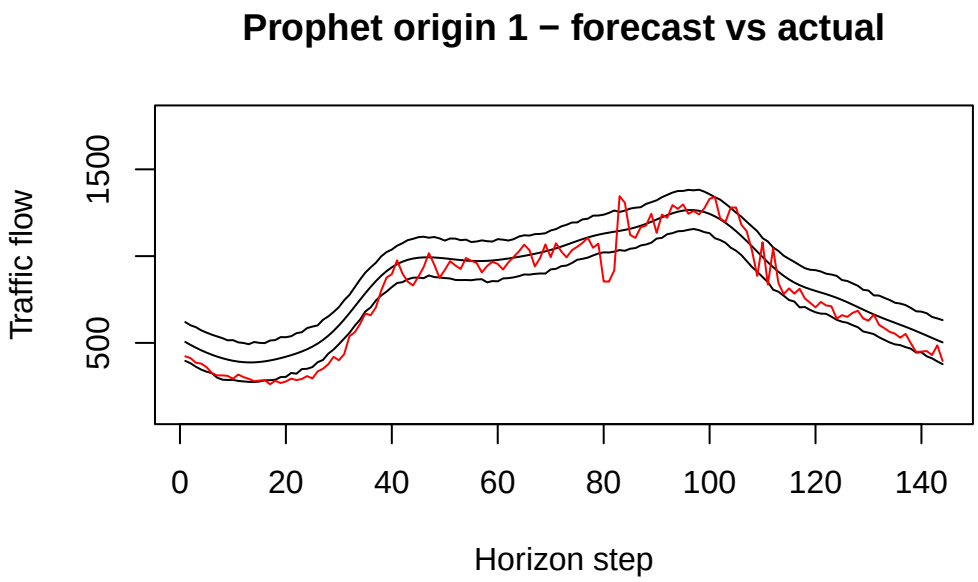

```
plot(prophet_pi$coverage_by_h, ylim = c(0, 1), xlab = "Forecast horizon", ylab = "Coverage")
abline(h = 0.95, col = "indianred")
```



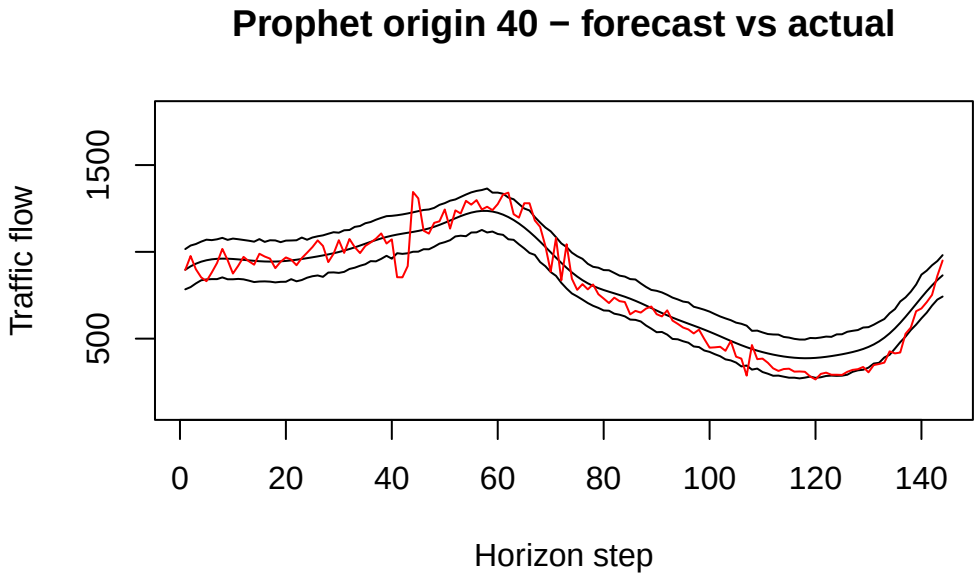
```r
plot(prophet_pi$width_by_h, xlab = "Forecast horizon", ylab = "PI width")
```




```r
plot(fc_prophet_full[1, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "T
     main = "Prophet origin 1 - forecast vs actual")
lines(fc_prophet_full_lower[1, ]); lines(fc_prophet_full_upper[1, ]); lines(actual_matrix_ful
```




```
plot(fc_prophet_full[40, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "
     main = "Prophet origin 40 - forecast vs actual")
lines(fc_prophet_full_lower[40, ]); lines(fc_prophet_full_upper[40, ]); lines(actual_matrix_
```



DHR

```
k_grid <- expand.grid(K1 = c(4, 6, 8, 10), K2 = c(2, 3, 4, 6))

dhr_results <- list()

for (i in 1:nrow(k_grid)) {
  K1 <- k_grid$K1[i]
  K2 <- k_grid$K2[i]

  time_dhr <- system.time({
    fit_dhr <- traffic_train %>%
      model(
        DHR = ARIMA(
          Traffic_flow ~
            fourier(period = 144,  K = K1) +
            fourier(period = 1008, K = K2) +
            PDQ(0,0,0)
        )
      )
  })

  fitted_dhr <- fitted(fit_dhr)$.fitted
  mae  <- mean(abs(y_train - fitted_dhr), na.rm = TRUE)
  rmse <- sqrt(mean((y_train - fitted_dhr)^2, na.rm = TRUE))
  aicc <- glance(fit_dhr)$AICc

  dhr_results[[i]] <- tibble(K1 = K1, K2 = K2, MAE = mae, RMSE = rmse,
                             AICc = aicc, time_sec = time_dhr["elapsed"])

  cat("K1=", K1, "K2=", K2, "MAE=", round(mae, 2), "AICc=", round(aicc, 1),
      "time=", round(time_dhr["elapsed"], 1), "s\n")
}
```

```
K1= 4 K2= 2 MAE= 30.3 AICc= 59707.6 time= 6.7 s
K1= 6 K2= 2 MAE= 29.56 AICc= 59464.8 time= 9.1 s
K1= 8 K2= 2 MAE= 29.22 AICc= 59405.7 time= 12.2 s
K1= 10 K2= 2 MAE= 29.21 AICc= 59402.7 time= 15.7 s
K1= 4 K2= 3 MAE= 30.29 AICc= 59710.4 time= 7.9 s
K1= 6 K2= 3 MAE= 29.52 AICc= 59467.8 time= 9.5 s
K1= 8 K2= 3 MAE= 29.2 AICc= 59408.1 time= 13.3 s
```

```
K1= 10 K2= 3 MAE= 29.2 AICc= 59405.1 time= 18 s
K1= 4 K2= 4 MAE= 30.29 AICc= 59713.3 time= 10.9 s
K1= 6 K2= 4 MAE= 29.52 AICc= 59470.2 time= 10.9 s
K1= 8 K2= 4 MAE= 29.19 AICc= 59410.4 time= 15.7 s
K1= 10 K2= 4 MAE= 29.19 AICc= 59407.4 time= 20.3 s
K1= 4 K2= 6 MAE= 30.19 AICc= 59690 time= 14.4 s
K1= 6 K2= 6 MAE= 29.38 AICc= 59433.3 time= 18.6 s
K1= 8 K2= 6 MAE= 28.99 AICc= 59372.9 time= 20.7 s
K1= 10 K2= 6 MAE= 28.98 AICc= 59369.4 time= 27 s
```

```
dhr_summary <- bind_rows(dhr_results)
dhr_summary %>% arrange(MAE) %>% print(n = Inf)
```

```
# A tibble: 16 x 6
      K1    K2   MAE  RMSE   AICc time_sec
   <dbl> <dbl> <dbl> <dbl>  <dbl>    <dbl>
 1    10     6  29.0  41.6 59369.    27.0
 2     8     6  29.0  41.7 59373.    20.7
 3    10     4  29.2  41.8 59407.    20.3
 4     8     4  29.2  41.8 59410.    15.7
 5    10     3  29.2  41.8 59405.    18.0
 6     8     3  29.2  41.8 59408.    13.3
 7    10     2  29.2  41.8 59403.    15.7
 8     8     2  29.2  41.8 59406.    12.2
 9     6     6  29.4  41.9 59433.    18.6
10     6     4  29.5  42.1 59470.    10.9
11     6     3  29.5  42.1 59468.     9.55
12     6     2  29.6  42.1 59465.     9.06
13     4     6  30.2  42.9 59690.    14.4
14     4     4  30.3  43.0 59713.    10.9
15     4     3  30.3  43.0 59710.     7.90
16     4     2  30.3  43.0 59708.     6.67
```

```
report(fit_dhr)
```

```
Series: Traffic_flow
Model: LM w/ ARIMA(2,1,3) errors

Coefficients:
         ar1      ar2      ma1     ma2      ma3
      0.9585  -0.9004  -1.4320  1.2775  -0.4363
```

```
s.e.  0.0139   0.0178   0.0189  0.0335   0.0194
      fourier(period = 144, K = K1)C1_144  fourier(period = 144, K = K1)S1_144
                                -313.3595                            -187.4560
s.e.                               7.7601                               7.7585
      fourier(period = 144, K = K1)C2_144  fourier(period = 144, K = K1)S2_144
                                 -52.1303                             -
15.5720
s.e.                               3.9046                               3.9066
      fourier(period = 144, K = K1)C3_144  fourier(period = 144, K = K1)S3_144
                                  57.8944                             -
19.4332
s.e.                               2.6379                               2.6394
      fourier(period = 144, K = K1)C4_144  fourier(period = 144, K = K1)S4_144
                                  9.2680                               -
3.6143
s.e.                               2.0147                               2.0159
      fourier(period = 144, K = K1)C5_144  fourier(period = 144, K = K1)S5_144
                                -26.8084                             12.7922
s.e.                               1.6486                               1.6495
      fourier(period = 144, K = K1)C6_144  fourier(period = 144, K = K1)S6_144
                                  1.4534                               -
0.2271
s.e.                               1.4106                               1.4113
      fourier(period = 144, K = K1)C7_144  fourier(period = 144, K = K1)S7_144
                                  6.5921                               -
8.1768
s.e.                               1.2456                               1.2462
      fourier(period = 144, K = K1)C8_144  fourier(period = 144, K = K1)S8_144
                                  1.4883                               1.0777
s.e.                               1.1260                               1.1265
      fourier(period = 144, K = K1)C9_144  fourier(period = 144, K = K1)S9_144
                                 -0.6372                               -
0.7341
s.e.                               1.0364                               1.0369
      fourier(period = 144, K = K1)C10_144
                                 -3.0377
s.e.                              0.9679
      fourier(period = 144, K = K1)S10_144
                                  0.9127
s.e.                              0.9682
      fourier(period = 1008, K = K2)C1_1008
                                  51.1901
s.e.                              54.8499
```

```
      fourier(period = 1008, K = K2)S1_1008
                                   -2.0666
s.e.                               53.5321
      fourier(period = 1008, K = K2)C2_1008
                                  -38.7469
s.e.                               27.1846
      fourier(period = 1008, K = K2)S2_1008
                                   13.7969
s.e.                               27.0567
      fourier(period = 1008, K = K2)C3_1008
                                   16.7051
s.e.                               18.0766
      fourier(period = 1008, K = K2)S3_1008
                                  -16.0701
s.e.                               18.1056
      fourier(period = 1008, K = K2)C4_1008
                                  -16.3182
s.e.                               13.5835
      fourier(period = 1008, K = K2)S4_1008
                                   -1.6137
s.e.                               13.5637
      fourier(period = 1008, K = K2)C5_1008
                                   40.8519
s.e.                               10.8561
      fourier(period = 1008, K = K2)S5_1008
                                    4.6144
s.e.                               10.8718
      fourier(period = 1008, K = K2)C6_1008
                                  -50.9765
s.e.                                9.0343
      fourier(period = 1008, K = K2)S6_1008
                                    9.8640
s.e.                                9.0764

sigma^2 estimated as 1744:  log likelihood=-29646.44
AIC=59368.88   AICc=59369.39   BIC=59621.9
```

```
needed <- c("h", "n_train", "y_full", "actual_matrix_full", "traffic_10min", "coverage_width"
sapply(needed, exists)
```

```
                h            n_train             y_full actual_matrix_full
             TRUE               TRUE               TRUE               TRUE
```

```
traffic_10min     coverage_width
         TRUE               TRUE
```

```
best_k <- dhr_summary %>% arrange(MAE) %>% slice(1)
cat("Winning K1 =", best_k$K1, " K2 =", best_k$K2, "\n")
```

```
Winning K1 = 10  K2 = 6
```

```
dhr_full_path <- "fc_dhr_full.RData"

if (file.exists(dhr_full_path)) {

  message("[cache hit] loading ", dhr_full_path, " - skipping DHR expanding window loop")
  load(dhr_full_path)

} else {

  n_workers <- max(1, parallel::detectCores() - 1)
  cat("Running DHR expanding window (", FORECAST_ORIGINS, "refits) across", n_workers, "worke

  plan(multisession, workers = n_workers)

  time_dhr_full <- system.time({
    results_dhr <- future_lapply(1:FORECAST_ORIGINS, function(i) {

      train_end_i <- n_train + i - 1
      train_i     <- traffic_10min %>% dplyr::slice(1:train_end_i)

      tryCatch({
        fit_i <- train_i %>%
          model(
            DHR = ARIMA(
              Traffic_flow ~
                fourier(period = 144,  K = 10) +
                fourier(period = 1008, K = 6) +
                pdq(2,1,3) +
                PDQ(0,0,0)
            )
          )

        fc_i   <- fit_i %>% forecast(h = h)
        hilo_i <- fabletools::hilo(fc_i, level = 95)
```

```
        list(
          mean  = fc_i$.mean,
          lower = hilo_i$`95%`$lower,
          upper = hilo_i$`95%`$upper,
          ok = TRUE, error = NA_character_
        )
      }, error = function(e) {
        list(mean = rep(NA_real_, h), lower = rep(NA_real_, h), upper = rep(NA_real_, h),
             ok = FALSE, error = conditionMessage(e))
      })

    }, future.seed = TRUE, future.packages = c("fable", "fabletools", "tsibble", "dplyr"))
  })

  plan(sequential)

  fc_dhr_full       <- do.call(rbind, lapply(results_dhr, `[[`, "mean"))
  fc_dhr_full_lower <- do.call(rbind, lapply(results_dhr, `[[`, "lower"))
  fc_dhr_full_upper <- do.call(rbind, lapply(results_dhr, `[[`, "upper"))

  n_failed <- sum(!sapply(results_dhr, `[[`, "ok"))
  if (n_failed > 0) {
    cat(n_failed, "origin(s) failed to fit - filled with NA.\n")
  } else {
    cat("All", FORECAST_ORIGINS, "origins fit successfully.\n")
  }
  cat("DHR loop time:", round(time_dhr_full["elapsed"]/60, 2), "minutes\n")

  save(fc_dhr_full, fc_dhr_full_lower, fc_dhr_full_upper, file = dhr_full_path)
  cat("DHR forecasts + 95% PI saved.\n")
}
```

```
[cache hit] loading fc_dhr_full.RData - skipping DHR expanding window loop
```

```
mae_dhr_full <- sapply(1:h, function(hh) {
  mean(abs(fc_dhr_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh]), na.rm = TRUE)
})
rmse_dhr_full <- sapply(1:h, function(hh) {
  sqrt(mean((fc_dhr_full[, hh] - actual_matrix_full[1:FORECAST_ORIGINS, hh])^2, na.rm = TRUE
})
cat("DHR MAE: ", round(mean(mae_dhr_full), 3), "\n")
```

```
DHR MAE:  96.767
```

```
cat("DHR RMSE:", round(mean(rmse_dhr_full), 3), "\n")
```

```
DHR RMSE: 121.75
```

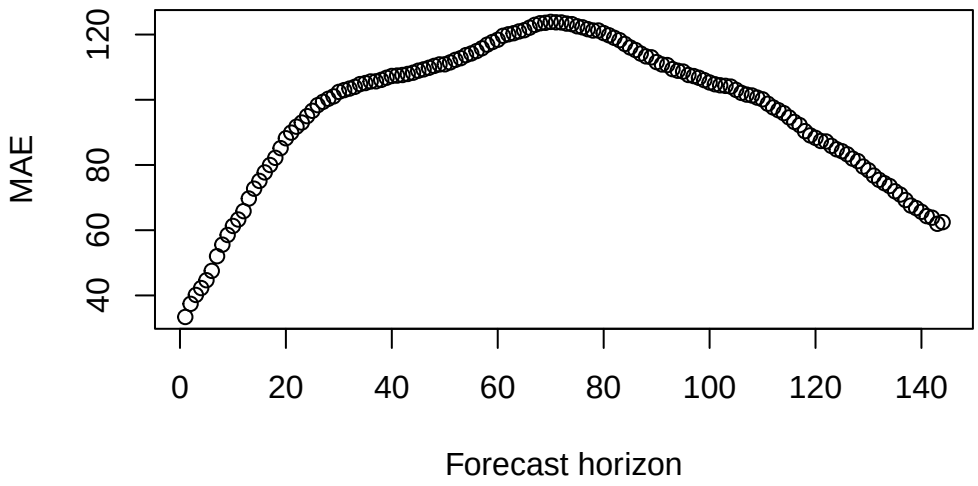
```
dhr_pi <- coverage_width(actual_matrix_full[1:FORECAST_ORIGINS, ], fc_dhr_full_lower, fc_dhr
cat("DHR 95% coverage:", round(dhr_pi$coverage_overall, 3), " (target 0.95)\n")
```

```
DHR 95% coverage: 0.956  (target 0.95)
```

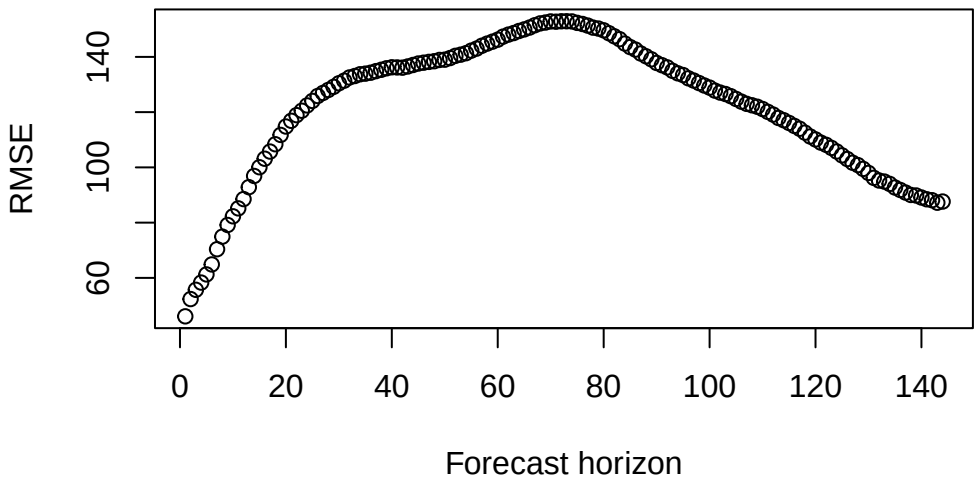
```
cat("DHR 95% width:   ", round(dhr_pi$width_overall, 2), "\n")
```

```
DHR 95% width:    597.61
```

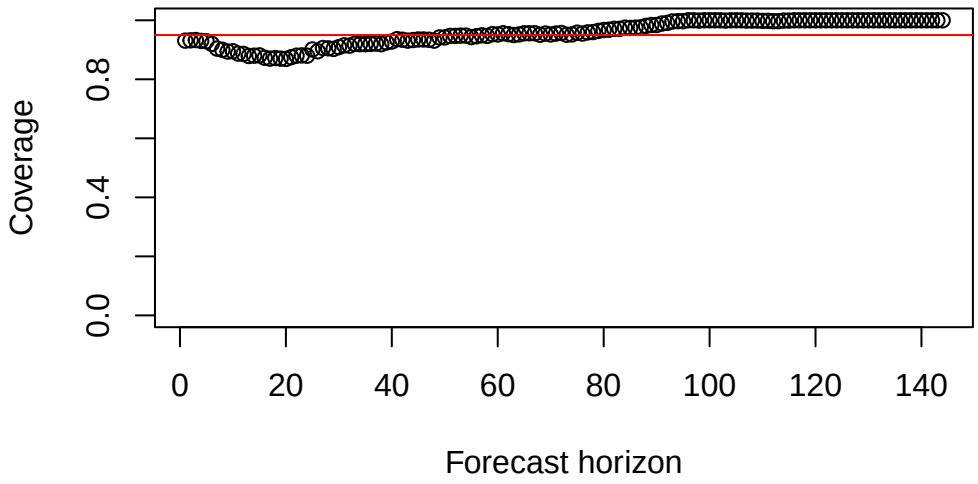
```
plot(mae_dhr_full, xlab = "Forecast horizon", ylab = "MAE")
```



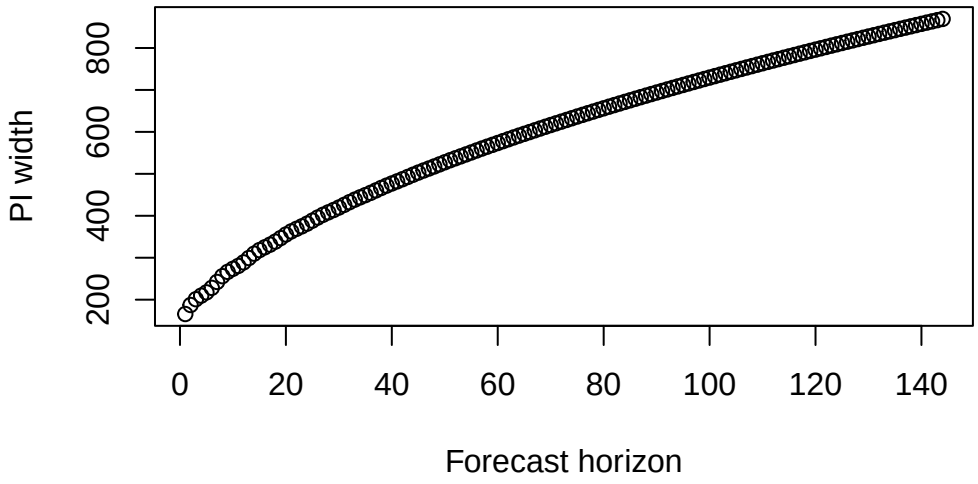

```
plot(rmse_dhr_full, xlab = "Forecast horizon", ylab = "RMSE")
```



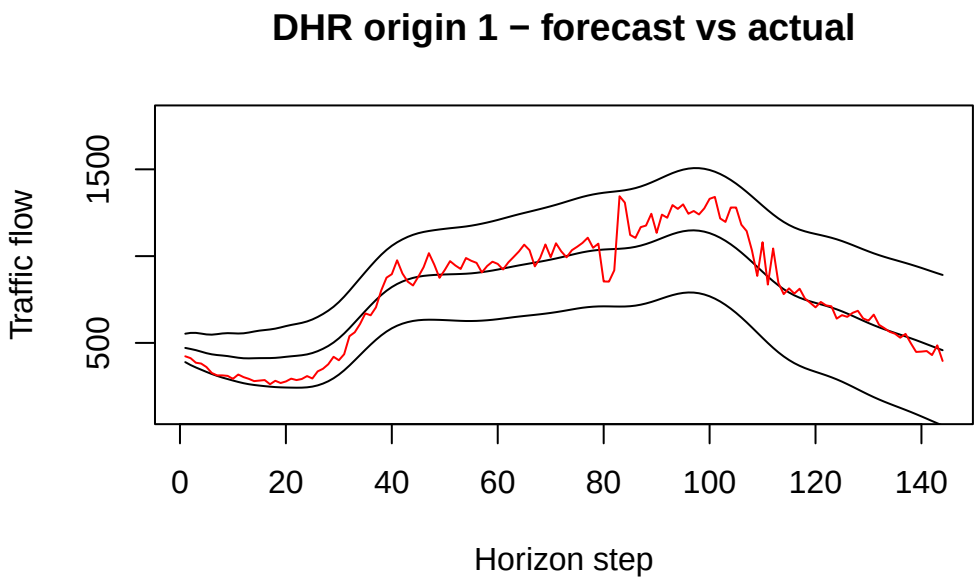

```
plot(dhr_pi$coverage_by_h, ylim = c(0, 1), xlab = "Forecast horizon", ylab = "Coverage")
abline(h = 0.95, col = "red")
```



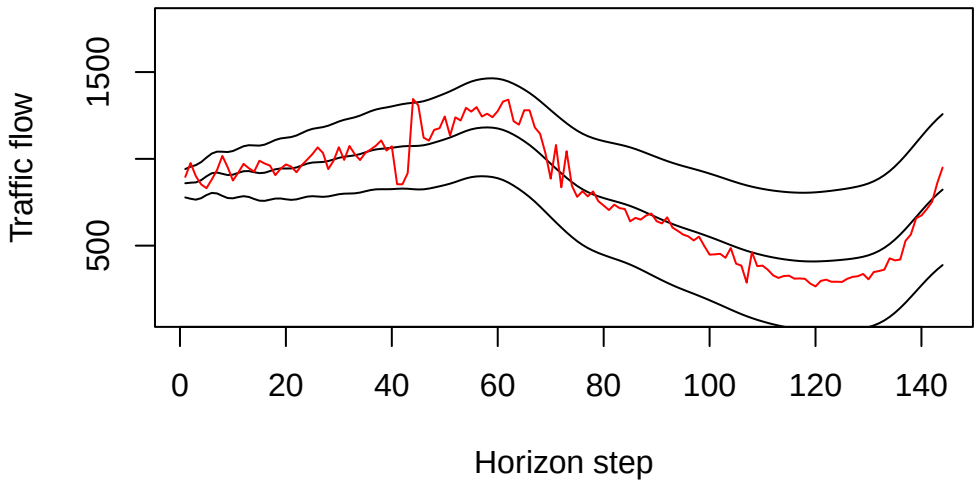
```r
plot(dhr_pi$width_by_h, xlab = "Forecast horizon", ylab = "PI width")
```




```r
# DHR
plot(fc_dhr_full[1, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Traff
     main = "DHR origin 1 - forecast vs actual")
lines(fc_dhr_full_lower[1, ]); lines(fc_dhr_full_upper[1, ]); lines(actual_matrix_full[1, ],
```




```
plot(fc_dhr_full[40, ], ylim = c(100, 1800), type = "l", xlab = "Horizon step", ylab = "Traf
     main = "DHR origin 40 - forecast vs actual")
lines(fc_dhr_full_lower[40, ]); lines(fc_dhr_full_upper[40, ]); lines(actual_matrix_full[40,
```



```r
comparison_table <- data.frame(
  Model    = c("mSARIMA", "SARIMA", "TBATS", "MSTL", "Prophet", "DHR"),
  MAE      = c(mean(mae_msarima_full), mean(mae_sarima_full), mean(mae_tbats_full),
               mean(mae_mstl_full),    mean(mae_prophet_full), mean(mae_dhr_full)),
  RMSE     = c(mean(rmse_msarima_full), mean(rmse_sarima_full), mean(rmse_tbats_full),
               mean(rmse_mstl_full),    mean(rmse_prophet_full), mean(rmse_dhr_full)),
  Coverage = c(msarima_pi$coverage_overall, sarima_pi$coverage_overall, tbats_pi$coverage_ove
               mstl_pi$coverage_overall,    prophet_pi$coverage_overall, dhr_pi$coverage_over
  Width    = c(msarima_pi$width_overall, sarima_pi$width_overall, tbats_pi$width_overall,
               mstl_pi$width_overall,    prophet_pi$width_overall, dhr_pi$width_overall)
)

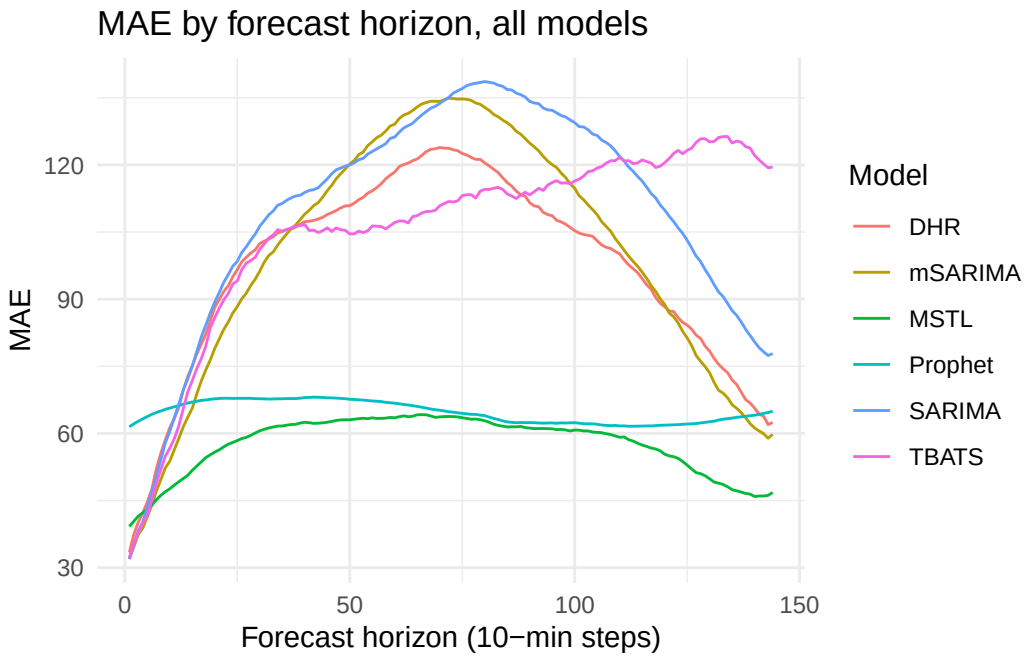
comparison_table <- comparison_table[order(comparison_table$RMSE), ]
print(comparison_table)
```

```
    Model       MAE      RMSE  Coverage    Width
4    MSTL  57.49531  73.96035 0.8855833 258.2604
5 Prophet  64.73022  78.85698 0.8889236 233.9515
1 mSARIMA  99.49069 121.16809 0.9723542 599.1154
6     DHR  96.76706 121.75041 0.9564583 597.6140
2  SARIMA 109.12891 139.60178 0.9655417 699.9844
3   TBATS 105.29250 144.79105 0.8661181 536.7442
```

```r
write.csv(comparison_table, "model_comparison_summary.csv", row.names = FALSE)
```

```r
horizon_df <- data.frame(
  h        = rep(1:h, 6),
  MAE      = c(mae_msarima_full, mae_sarima_full, mae_tbats_full,
               mae_mstl_full, mae_prophet_full, mae_dhr_full),
  Model    = rep(c("mSARIMA","SARIMA","TBATS","MSTL","Prophet","DHR"), each = h)
)

library(ggplot2)
ggplot(horizon_df, aes(x = h, y = MAE, color = Model)) +
  geom_line() +
  labs(x = "Forecast horizon (10-min steps)", y = "MAE", title = "MAE by forecast horizon, al
  theme_minimal()
```




```
for (f in c("fc_msarima_full.RData", "fc_msarima_full1.RData",
            "fc_sarima_full.RData",
            "fc_tbats_full.RData",
            "fc_mstl_full.RData",
            "fc_prophet_full.RData",
            "fc_dhr_full.RData")) {
  cat(f, ":", file.exists(f), "\n")
}
```

```
fc_msarima_full.RData : TRUE
fc_msarima_full1.RData : FALSE
fc_sarima_full.RData : TRUE
fc_tbats_full.RData : TRUE
fc_mstl_full.RData : TRUE
fc_prophet_full.RData : TRUE
fc_dhr_full.RData : TRUE
```